Llama 模型

第 4 章

Llama 模型 · 模型结构 · 键值缓存 · 长文本继续预训练

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

4.1 **Llama 模型** 第 3 页
开源里程碑与预训练规模

4.2 **Llama 模型结构** 第 14 页
整体结构、Llama 块与四个关键模块

4.3 **键值缓存** 第 44 页
生成分析、实现与显存

4.4 **长文本继续预训练** 第 58 页
扩展方案与位置编码改造

4.5 **小结** 第 77 页
结构要点与后续衔接

4.6 **附录：稀疏混合专家系统** 第 80 页
Top-K 门控与负载均衡

阅读建议 先读 Llama 的模型结构与各模块（4.2），再看 KV-Cache 与长文本扩展（4.3–4.4）。

4.1

Llama 模型

从 65B 首次超越 GPT-3，到用 15 万亿词元训练的 Llama-3：先看这一系列为何成为开源大模型的基石。

本节内容

4.1.1 模型介绍

4.1.2 Llama 模型预训练

Llama 模型概述

Llama（Large Language Model Meta AI）是 Meta 于 2023 年 2 月发布的大语言模型系列，参数规模覆盖 70 亿（7B）到 650 亿（65B）。

2023.02
首个版本发布

7B–65B
参数规模覆盖

65B / 175B
首次在多个基准上超越 GPT-3

- 发布之时，开源大语言模型在模型规模、性能和工具生态方面普遍落后于领先的闭源模型。
- Llama 65B 在多个基准测试上的表现首次超越 GPT-3（175B），成为高性能开源大模型发展的重要里程碑。
- Llama-2 与 Llama-3 采用相对宽松的开源协议，在特定条件下允许免费商用，推动了全球开发者生态的爆发式增长。

依据 《大语言模型：训练、精调与对齐》第 4.1.1 节

Llama 的开源生态

Llama 的开源让研究机构与中小企业能以极低成本获得接近 GPT-3.5 的能力；围绕其架构迅速生长出一批衍生模型。

中文增强

Chinese-LLaMA

继续预训练增强中文能力。

代码

CodeLlama

原生 16K 上下文，可继续扩展。

指令微调

Alpaca-7B

Self-Instruct 数据微调，指令遵循接近 text-davinci-003。

对话

Vicuna

基于公开的 ShareGPT 对话数据。

多模态

LLaVA

在 Vicuna 上扩展到图文多模态。

垂直领域

金融 · 医疗 · 安全

FinGPT、HuatuoGPT、LlamaGuard。

Llama 的工具链

Llama 的核心价值不仅在模型本身，还在于围绕它形成的工程最佳实践：主流框架均对其提供良好支持。

训练与微调

Transformers

DeepSpeed 提供显存优化与分布式训练；LlamaFactory 统一微调流程。

推理

llama.cpp · vLLM

覆盖本地量化推理与高吞吐服务；TensorRT-LLM 面向 GPU 上的极致性能。

应用开发

LangChain · LlamaIndex

支撑检索增强、企业级知识库问答等应用的产品化落地。

> 注 大量前沿研究以 Llama 系列为基础展开，框架、数据、评测与衍生模型共同构成了完整的开源技术栈。

Llama 的四项技术改进

Llama-3 的训练语料达到 15 万亿词元，并在模型规模与数据配比上遵循并优化了 Chinchilla 缩放定律；结构上有四项关键改动。

4.2.3

均方根归一化

以 RMSNorm 替代层归一化，省去均值计算，加速收敛。

4.2.4

分组查询注意力

GQA 让多组查询头共享键值头，优化大模型推理效率。

4.2.5

旋转位置编码

RoPE 在注意力中注入相对位置，增强长序列依赖建模。

4.2.6

SwiGLU 激活

门控前馈层替代标准激活，提升模型表示能力。

生态层面还广泛采用继续预训练扩展上下文长度（4.4），并配合有监督微调与基于人类反馈的强化学习，提升指令遵循、有用性、无害性与安全性。

预训练数据

训练高性能语言模型的关键之一，是筛选大规模高质量的训练数据集；Llama 系列模型的预训练数据均来自公开渠道。

1.4T

Llama-1 词元

2T

Llama-2 词元

15T

Llama-3 词元

语料由七类公开数据构成，经 BPE 训练得到 32 000 词元的分词器。

数据来源	内容
CommonCrawl	网页爬取文本，主要选取英文数据
C4	大规模高质量网页文本
Github	开源代码数据
Wikipedia	多语种百科内容
Books	Gutenberg 项目中可用的书本级数据
ArXiv	预印本论文
StackExchange	多个专业工科领域的问答内容

依据 《大语言模型：训练、精调与对齐》第 4.1.2 节

数据清洗与配比

从 1.4T 到 15T，不只是放大规模：Llama-3 的数据集约为 Llama-2 的 7.5 倍，代码数据达到 4 倍，非英语数据不超过 5%，却覆盖 30 余种语言。



- 清洗手段包括启发式过滤器、不适内容过滤器、语义去重，以及用于评估数据质量的文本分类器。
- 通过大量实验评估不同来源数据在最终预训练集中的最优混合比例，这一配比让 Llama-3 在常识问答、STEM、编程、历史知识等场景中均表现突出。
- Llama-3 的训练策略是先在 8192 词元上下文上预训练，再通过继续预训练把长度扩展到 128k（详见 4.4）。

依据 《大语言模型：训练、精调与对齐》第 4.1.2 节

可复现的替代语料

Llama 的预训练数据集本身并未开源。为了让训练过程可复现，多个开源项目公开了同量级的语料与处理流程。

2T 词元

RedPajama

公开约 2 万亿词元规模的预训练语料。

15T 词元

FineWeb

公开约 15 万亿词元的网页语料。

200T → 2T

DCLM

公开从 200 万亿词元清洗过滤到 2 万亿词元的处理流程与数据集。

全流程开源

OLMo

基于 DCLM 数据预训练，开源从数据到训练的全流程。

Llama 系列的参数配置

表 4.1　Llama 系列模型参数信息

名称	发布日期	参数	上下文长度	词表大小	预训练语料库
Llama-1	2023.02	6.7B, 13B, 32.5B, 65.2B	2048	32 000	1.4T
Llama-2	2023.07	6.7B, 13B, 70B	4096	32 000	2.0T
Llama-3	2024.04	8B, 70.6B	128 000	128 256	15T
Llama-4 Scout	2025.04	17B×16E	10M	128 256	~40T
Llama-4 Maverick	2025.04	17B×128E	1M	128 256	~22T
Llama-4 Behemoth	—	—	—	—	—

- Llama-1、2、3 系列均采用密集（Dense）型纯解码器 Transformer 架构，参数规模最大达到 70B。
- Llama-4 系列改为混合专家（MoE）型纯解码器架构，参数量以“17B×16E”的形式标注（见 4.6 节）。

依据 《大语言模型：训练、精调与对齐》第 4.1.2 节（表 4.1）

小模型追平大模型

Llama-3 的 8B 模型在 15 万亿词元上完成训练后，性能达到 Llama-2 70B 的水平，远超 Chinchilla 缩放定律给出的最优训练量。

8B

追平 Llama-2 70B

75×

相对 Chinchilla 最优值

8192 → 128k

上下文长度

- 15T 语料主要来自互联网数据，经过隐私与安全过滤、文本提取与清洗、去重、启发式过滤、领域标注、多语言处理与模型过滤。
- 训练量超出缩放定律估算的最优值 75 倍而仍有收益，说明语言模型的规模扩展潜力还有广阔的探索空间。
- 长上下文不是一次预训练得到：先在 8192 上预训练，再通过继续预训练逐步扩展到 128k，显著增强超长文本理解能力。

训练系统与 MoE 架构

算力与并行

16 000 块 H100

Llama-3 的预训练在包含 16 000 块 H100 的集群上进行，依赖分布式混合并行策略（见第 9 章）。

405B 模型在训练周期内共发生 466 次中断，其中 78% 由硬件故障引起。

容灾

系统工程同样关键

超大规模训练既需要高效的混合并行策略，也需要具备强大容灾恢复能力的高可靠基础设施与系统工程体系。

Llama-4 · 混合专家

17B×16E 的含义

模型包含 16 个独立专家，每个专家规模为 170 亿参数；MoE 通过路由机制在每次推理时仅稀疏激活少量专家，兼顾模型容量与推理效率。DeepSeek-V3 以 671B 总参数展示了这一范式的潜力，MoE 已成为主流架构之一。

4.2

Llama 模型结构

纯解码器架构不变，但四个模块被替换：RMSNorm 取代层归一化，GQA 取代 MHA，RoPE 取代绝对位置编码，SwiGLU 取代标准前馈层。

本节内容

4.2.1 Llama 整体结构

4.2.2 Llama 块

4.2.3 均方根归一化

4.2.4 分组查询注意力机制

4.2.5 旋转位置编码

4.2.6 SwiGLU

Llama 的三个组成部分

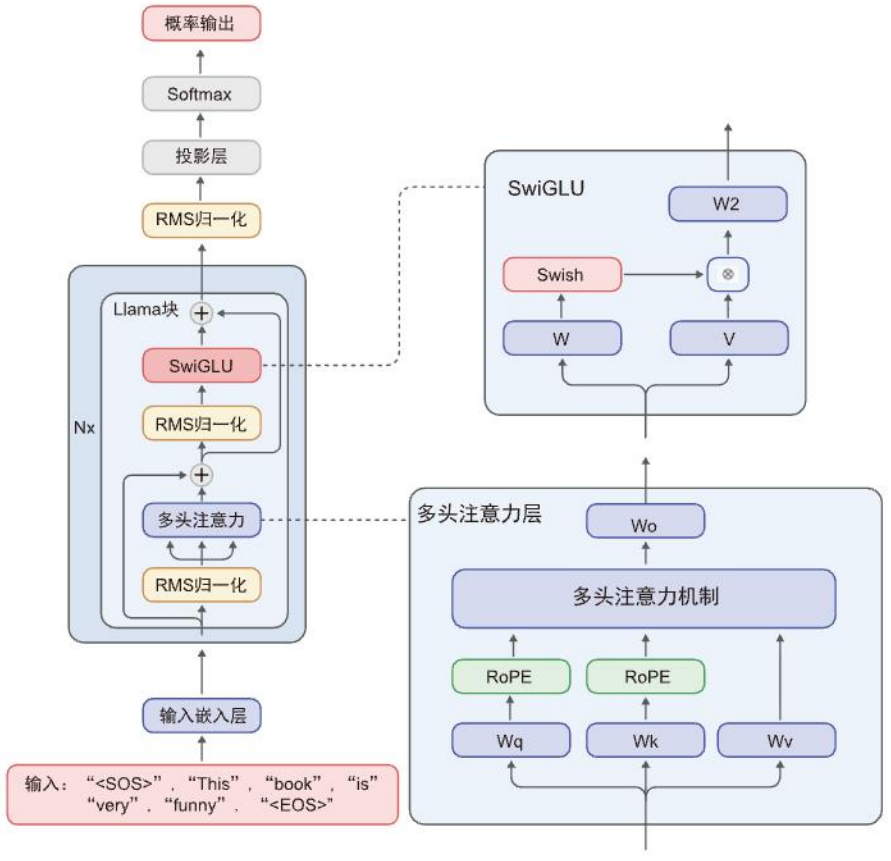


图 4.1　Llama 架构图

Llama 是基于纯解码器 Transformer 的模型，预训练目标与 GPT-2 相同，都是下一个词元预测。

输入层

只有嵌入层

没有位置编码层；位置由 RoPE 融入查询与键向量。

解码器

多个 Llama 块堆叠

每块含 RMSNorm、SwiGLU 前馈、RoPE 与多头注意力；GQA 仅见于部分大模型。

输出层

归一化 → 投影 → Softmax

先 RMSNorm，再投影到词表维度，经 Softmax 得到概率分布。

Llama 相对 Transformer 的两处简化

与 Transformer、GPT 类模型相比，Llama 做了两处一致的简化，都指向更少的参数与更稳定的数值。

位置信息：不再相加，改为旋转

绝对位置编码在输入层把 $\boldsymbol{p}_m$ 加到词嵌入 $\boldsymbol{x}_m$ 上，位置信息只注入一次。

Llama 移除该层，在每个注意力层内对 $\boldsymbol{q}$、$\boldsymbol{k}$ 施加 RoPE，位置信息随层反复注入（4.2.5）。

线性层：普遍移除偏置

$\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v, \boldsymbol{W}_o$ 与前馈层的线性变换都不带偏置 $\boldsymbol{b}$。

简化结构、减少参数，**避免学习预训练任务的固定偏置**。

词表

Llama-1/2 为 32 000，Llama-3 扩展到 128 256。词表越大嵌入层参数越多，小模型的存储开销主要来自这里。

块表示

每个 Llama 块记为 $\boldsymbol{X}^{(l)} = \texttt{Llama_Block}_l\left(\boldsymbol{X}^{(l-1)}\right)$，$\boldsymbol{X}^{(0)}$ 为解码器输入。

Llama 解码块的四个模块

$$\boldsymbol{X}^{(l)} = \texttt{Llama_Block}_l\left(\boldsymbol{X}^{(l-1)}\right) \tag{4-1}$$

$\boldsymbol{X}^{(l)}, \boldsymbol{X}^{(l-1)} \in \mathbb{R}^{n\times d}$，$n$ 为序列长度，d 为特征维度；块内各模块输入输出维度一致。

替代层归一化 · 4.2.3

$\texttt{RMSNorm}(\cdot)$

只做重缩放、不中心化，输入尺度不变，梯度更稳、计算更省。

替代 MHA · 4.2.4

$\texttt{GQA}(\cdot)$

查询头分组，组内共享键头与值头；性能接近 MHA，KV-Cache 显著减少。

注入位置 · 4.2.5

RoPE

在注意力前对 $\boldsymbol{q}$、$\boldsymbol{k}$ 施加旋转，分数只含相对位置。

替代前馈激活 · 4.2.6

$\texttt{SwiGLU}(\cdot)$

用 Swish 替代 GELU，为每个特征学习动态门控，增强重要特征。

解码块的两个子层

$$\boldsymbol{X}_{\text{norm1}} = \texttt{RMSNorm_1}\left(\boldsymbol{X}^{(l-1)}\right) \tag{4-2}$$

$$\boldsymbol{X}_{\text{attn}} = \texttt{Grouped_Query_Attention}\left(\boldsymbol{X}_{\text{norm1}}, \boldsymbol{M}\right) \tag{4-3}$$

$$\boldsymbol{X} = \boldsymbol{X}_{\text{attn}} + \boldsymbol{X}^{(l-1)} \qquad \text{(残差连接 1)} \tag{4-4}$$

$$\boldsymbol{X}_{\text{norm2}} = \texttt{RMSNorm_2}\left(\boldsymbol{X}\right) \tag{4-5}$$

$$\boldsymbol{X}_{\text{swiglu}} = \texttt{SwiGLU}\left(\boldsymbol{X}_{\text{norm2}}\right) \tag{4-6}$$

$$\boldsymbol{X}^{(l)} = \boldsymbol{X}_{\text{swiglu}} + \boldsymbol{X} \qquad \text{(残差连接 2)} \tag{4-7}$$

$\boldsymbol{M}$ 为注意力掩码；`Grouped_Query_Attention` 内部，查询向量与键向量已融入 RoPE 位置信息。

- 归一化放在子层之前（Pre-Norm）：注意力与前馈的输入都先经 RMSNorm，输出再与子层输入相加。
- 两条残差保证维度不变，块可以逐层堆叠；每层的 $\boldsymbol{X}$ 既是残差来源，也是下一子层的输入。

依据 《大语言模型：训练、精调与对齐》第 4.2.2 节（式 (4-2)–(4-7)）

RMSNorm 的定义

层归一化能处理变长序列、稳定训练，但它的统计量里含均值与方差。T5 已尝试去掉中心化与平移参数 $\boldsymbol{\beta}$；RMSNorm 沿这条路走到底。

$$\mathrm{RMS}(\boldsymbol{x}_i) = \mathrm{RMS}_i = \sqrt{\frac{1}{d}\sum_{j=1}^{d} x_{ij}^2} \tag{4-8}$$

$$\hat{\boldsymbol{x}}_i = \frac{\boldsymbol{x}_i}{\mathrm{RMS}_i} \tag{4-9}$$

$$\tilde{x}_{ij} = \gamma_j \hat{x}_{ij} \tag{4-10}$$

$\boldsymbol{x}_i \in \mathbb{R}^d$ 为第 i 个词元的特征向量，$\boldsymbol{\gamma} \in \mathbb{R}^d$ 为可学习缩放向量。不计算均值与方差，只用均方根 RMS_i 作缩放因子。

- 缩放后的 $\hat{\boldsymbol{x}}_i$ 满足 $\mathrm{RMS}(\hat{\boldsymbol{x}}_i) = 1$，即统计意义上的单位均方根值。
- 对比：层归一化把数据变成均值 0、标准差 1；RMSNorm 只保证尺度为 1，中心位置保持不变。

依据 《大语言模型：训练、精调与对齐》第 4.2.3 节（式 (4-8)–(4-10)）

去掉中心化的依据

RMSNorm 的核心观点是：层归一化的成功主要来自重缩放不变性，而不是重中心化不变性——只保留重缩放就足以达到相当的性能。

优势一

计算效率更高

层归一化要算均值与标准差；RMSNorm 去掉均值与 β，只求一个均方根作缩放因子。

原论文实测比层归一化快 10%–30%。

优势二

避免消除输入固有偏置

强制中心化隐含“输入应以零为中心”，但激活常天然非零均值，β 未必能重建被移除的结构。

不重中心化即保留原始均值信息，也避免预训练学到不利于下游迁移的偏置。

实测 在机器翻译、文本分类、语言建模等任务上，RMSNorm 与层归一化性能相当或略有提升，例如 WMT14 英德翻译 BLEU 提升 0.3。

层归一化与 RMSNorm 的逐步对照

两者共用同一条主线"除以一个统计量再乘缩放"：差别只在于统计量里有没有均值，以及有没有平移参数 $\boldsymbol{\beta}$。

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij}$$

$$\sigma_i = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(x_{ij}-\mu_i)^2}$$

$$\hat{x}_{ij} = \frac{x_{ij}-\mu_i}{\sigma_i}$$

$$\tilde{x}_{ij} = \gamma_j \hat{x}_{ij} + \beta_j$$

层归一化：先算均值 μ_i 与标准差 σ_i，中心化后再缩放，最后加回平移 β_j。两步统计量、两组参数。

$$\mathrm{RMS}_i = \sqrt{\frac{1}{d}\sum_{j=1}^{d} x_{ij}^2}$$

$$\hat{\boldsymbol{x}}_i = \frac{\boldsymbol{x}_i}{\mathrm{RMS}_i}$$

$$\tilde{x}_{ij} = \gamma_j \hat{x}_{ij}$$

RMSNorm：只用均方根 RMS_i 当分母，不减均值、不加 β_j。一个统计量、一组参数。

少一次求均值

先算 μ_i 再算 σ_i，变成一步开方。

少一组参数

省下平移向量 $\boldsymbol{\beta}$ 的 d 个参数。

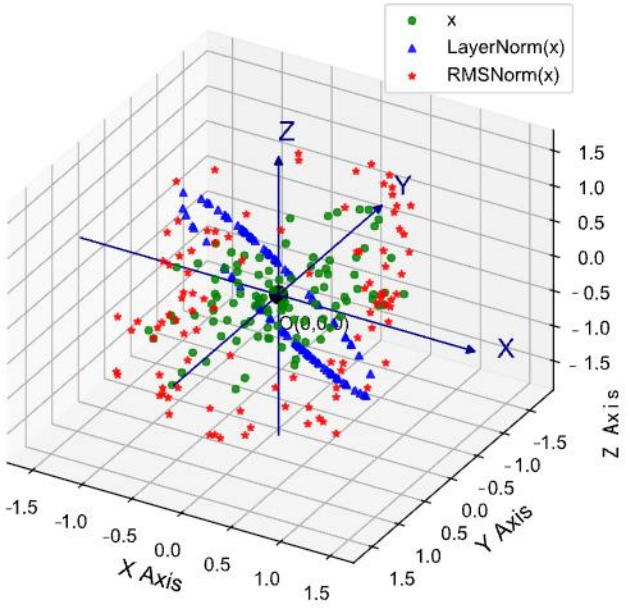
少一条不变性

不再对输入平移不变，保留中心位置。

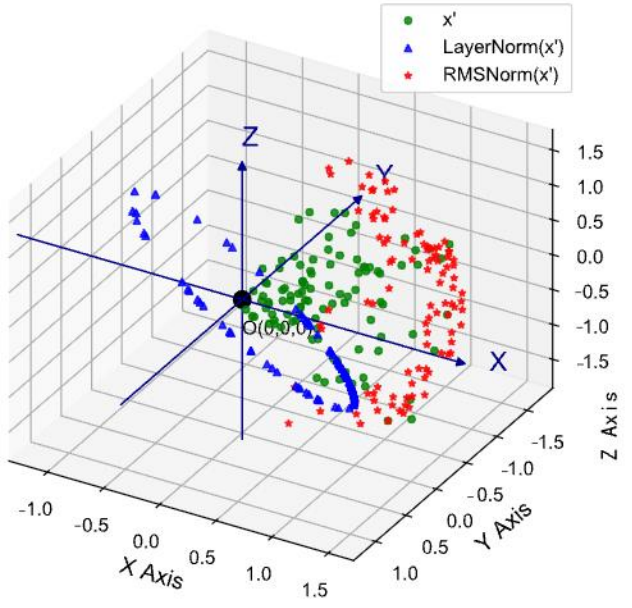
当 $\mu_i = 0$ 且 $\boldsymbol{\beta} = 0$ 时，σ_i 退化为 RMS_i，两式重合：RMSNorm 只保留重缩放不变性。

依据 《大语言模型：训练、精调与对齐》第 4.2.3 节；式 (4-8)–(4-10)

缩放对输出的影响



(a) 原始数据 $\boldsymbol{X}$



(b) 变换数据 $\boldsymbol{X}'$

- 设定：$\boldsymbol{X} \in \mathbb{R}^{n\times 3}$（$n = 100$）服从正态分布，$\boldsymbol{\gamma}$ 全 1、$\boldsymbol{\beta}$ 全 0。
- 图 (a) 对比原始、层归一化与 RMSNorm 的结果；图 (b) 是第 1、2 维取绝对值后的变换。
- RMSNorm 仅靠缩放保留输入结构（含固有中心位置）；层归一化则把中心强行移到原点。

RMSNorm 的输入尺度不变性

尺度不变性衡量归一化后的输出是否随输入或权重的缩放而剧烈变化。先写出变换函数，再把 $\alpha\boldsymbol{x}$ 代进去：

$$f(\boldsymbol{x}) = \boldsymbol{\gamma}\frac{\boldsymbol{x}}{\mathrm{RMS}(\boldsymbol{x})} \tag{4-11}$$

$\boldsymbol{\gamma} \in \mathbb{R}^d$ 为可学习缩放向量；忽略偏置，f 就是 RMSNorm 的前向。

$$\mathrm{RMS}(\alpha\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(\alpha x_j)^2} = \alpha\,\mathrm{RMS}(\boldsymbol{x}) \tag{4-12}$$

$\alpha > 0$ 可从根号内提出，故 RMS 随输入同比例缩放。

$$f(\alpha\boldsymbol{x}) = \boldsymbol{\gamma}\frac{\alpha\boldsymbol{x}}{\mathrm{RMS}(\alpha\boldsymbol{x})} = \boldsymbol{\gamma}\frac{\alpha\boldsymbol{x}}{\alpha\,\mathrm{RMS}(\boldsymbol{x})} = \boldsymbol{\gamma}\frac{\boldsymbol{x}}{\mathrm{RMS}(\boldsymbol{x})} = f(\boldsymbol{x}) \tag{4-13}$$

分子分母的 α 相消，输出与原输入完全一致：$f(\alpha\boldsymbol{x}) = f(\boldsymbol{x})$，RMSNorm 具有输入尺度不变性。

依据 《大语言模型：训练、精调与对齐》第 4.2.3 节（式 (4-11)–(4-13)）

RMSNorm 的权重尺度不变性

考虑无偏置线性层 $\boldsymbol{x} = \boldsymbol{v}\boldsymbol{W}$（$\boldsymbol{v}$ 为输入向量，$\boldsymbol{W}$ 为权重矩阵）。当权重缩放 α 倍时，其输出同样只被缩放，接上 RMSNorm 后缩放被抵消：

$$\boldsymbol{x}' = \boldsymbol{v}\,(\alpha\boldsymbol{W}) = \alpha\,(\boldsymbol{v}\boldsymbol{W}) = \alpha\boldsymbol{x} \tag{4-14}$$

α 是标量，可与矩阵乘法交换位置：权重缩放等价于该层输出的整体缩放。

$$f(\boldsymbol{x}') = f(\alpha\boldsymbol{x}) = f(\boldsymbol{x}) = f(\boldsymbol{v}\boldsymbol{W}) \tag{4-15}$$

第一步代入式 (4-14)，第二步用输入尺度不变性式 (4-13)。

- $f\left(\boldsymbol{v}(\alpha\boldsymbol{W})\right) = f(\boldsymbol{v}\boldsymbol{W})$：RMSNorm 对前接线性层的权重同样具有尺度不变性。
- 这意味着权重的整体缩放不会改变归一化后的表示，训练中不必依赖精细的权重初始化尺度。

依据 《大语言模型：训练、精调与对齐》第 4.2.3 节（式 (4-14)、(4-15)）

输入梯度的推导

按链式法则，反传到输入的梯度先分解成两个因子；雅可比用商的求导法则展开，再代入 $\mathrm{RMS}(\boldsymbol{x})$ 的导数。

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{x}}} \frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{x}} \tag{4-16}$$

损失对输入的总梯度 = 损失对输出的梯度 × RMSNorm 的雅可比。

$$\frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{x}} = \boldsymbol{\gamma} \frac{\partial}{\partial \boldsymbol{x}} \left(\frac{\boldsymbol{x}}{\mathrm{RMS}(\boldsymbol{x})} \right) \tag{4-17}$$

$$\frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{x}} = \boldsymbol{\gamma} \left(\frac{1}{\mathrm{RMS}(\boldsymbol{x})} - \frac{\boldsymbol{x} \dfrac{\partial \mathrm{RMS}(\boldsymbol{x})}{\partial \boldsymbol{x}}}{\mathrm{RMS}(\boldsymbol{x})^2} \right) \tag{4-18}$$

商的求导法则：分母平方，分子为“分母乘分子导数 − 分子乘分母导数”。

$$\frac{\partial \mathrm{RMS}(\boldsymbol{x})}{\partial \boldsymbol{x}} = \frac{\boldsymbol{x}}{d\,\mathrm{RMS}(\boldsymbol{x})}$$

由式 (4-8) 对 $\boldsymbol{x}$ 求导，是式 (4-18) 里唯一缺的因子。

$$\frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{x}} = \boldsymbol{\gamma} \left(\frac{1}{\mathrm{RMS}(\boldsymbol{x})} - \frac{\boldsymbol{x} \dfrac{\boldsymbol{x}}{d\,\mathrm{RMS}(\boldsymbol{x})}}{\mathrm{RMS}(\boldsymbol{x})^2} \right) \tag{4-19}$$

把 RMS 的导数代回式 (4-18) 分子的第二个因式。

$$\frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{x}} = \boldsymbol{\gamma} \left(\frac{1}{\mathrm{RMS}(\boldsymbol{x})} - \frac{\boldsymbol{x}^2}{d\,\mathrm{RMS}(\boldsymbol{x})^3} \right) \tag{4-20}$$

把 $\mathrm{RMS}(\boldsymbol{x})$ 乘进分母，得到 $d\,\mathrm{RMS}(\boldsymbol{x})^3$。

第一项是单纯的缩放项 $1/\mathrm{RMS}(\boldsymbol{x})$，第二项是均方根带来的修正；比层归一化的梯度少了“减去均值”对应的那一项，结构更简单。

梯度的尺度不变性

把输入缩放 α 倍，逐项代入式 (4-20)，看梯度如何变化。

$$\frac{\partial \tilde{\boldsymbol{x}}}{\partial(\alpha \boldsymbol{x})} = \boldsymbol{\gamma}\left(\frac{1}{\mathrm{RMS}(\alpha \boldsymbol{x})} - \frac{\alpha^2 \boldsymbol{x}^2}{d\,\mathrm{RMS}(\alpha \boldsymbol{x})^3}\right) \tag{4-21}$$

$$= \boldsymbol{\gamma}\left(\frac{1}{\alpha\,\mathrm{RMS}(\boldsymbol{x})} - \frac{\alpha^2 \boldsymbol{x}^2}{d\,\alpha^3 \mathrm{RMS}(\boldsymbol{x})^3}\right) \tag{4-22}$$

$$= \boldsymbol{\gamma}\frac{1}{\alpha}\left(\frac{1}{\mathrm{RMS}(\boldsymbol{x})} - \frac{\boldsymbol{x}^2}{d\,\mathrm{RMS}(\boldsymbol{x})^3}\right) \tag{4-23}$$

$$= \frac{1}{\alpha}\frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{x}} \tag{4-24}$$

式 (4-22)：$\mathrm{RMS}(\alpha \boldsymbol{x}) = \alpha \mathrm{RMS}(\boldsymbol{x})$，且分子含 α^2、分母含 α^3；式 (4-23)：提出公共因子 $1/\alpha$。

- 输入梯度随缩放因子 α 成反比变化：输入越大，梯度越小。
- 这种自动反向调节有效抑制了梯度爆炸与消失。

依据 《大语言模型：训练、精调与对齐》第 4.2.3 节（式 (4-21)–(4-24)）

参数梯度的尺度不变性

再看前接线性层 $\boldsymbol{x} = \boldsymbol{v}\boldsymbol{W}$（无偏置）的权重梯度。当输入缩放 α 倍时，按链式法则逐层相乘：

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}} = \frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{x}}} \frac{\partial \tilde{\boldsymbol{x}}}{\partial (\alpha \boldsymbol{v}\boldsymbol{W})} \frac{\partial (\alpha \boldsymbol{v}\boldsymbol{W})}{\partial \boldsymbol{W}} \tag{4-25}$$

$$= \frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{x}}} \left(\frac{1}{\alpha} \frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{v}\boldsymbol{W}} \right) (\alpha \boldsymbol{v}) \tag{4-26}$$

$$= \frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{x}}} \frac{\partial \tilde{\boldsymbol{x}}}{\partial \boldsymbol{x}} \boldsymbol{v} \tag{4-27}$$

式 (4-26)：由式 (4-24) 得 $\partial \tilde{\boldsymbol{x}} / \partial (\alpha \boldsymbol{x}) = \frac{1}{\alpha} \partial \tilde{\boldsymbol{x}} / \partial \boldsymbol{x}$；对 $\boldsymbol{W}$ 求导得 $\alpha \boldsymbol{v}$。式 (4-27)：$\frac{1}{\alpha} \cdot \alpha = 1$，代回 $\boldsymbol{x} = \boldsymbol{v}\boldsymbol{W}$。

- 对比原始梯度 $\partial \mathcal{L} / \partial \boldsymbol{W} = (\partial \mathcal{L} / \partial \tilde{\boldsymbol{x}})(\partial \tilde{\boldsymbol{x}} / \partial \boldsymbol{x}) \boldsymbol{v}$，缩放因子 α 被完全抵消。
- 由此，RMSNorm 具有参数梯度尺度不变性：权重的整体缩放不改变梯度，训练对学习率与初始化的尺度不敏感。

依据 《大语言模型：训练、精调与对齐》第 4.2.3 节（式 (4-25)–(4-27)）

分组查询注意力（GQA）

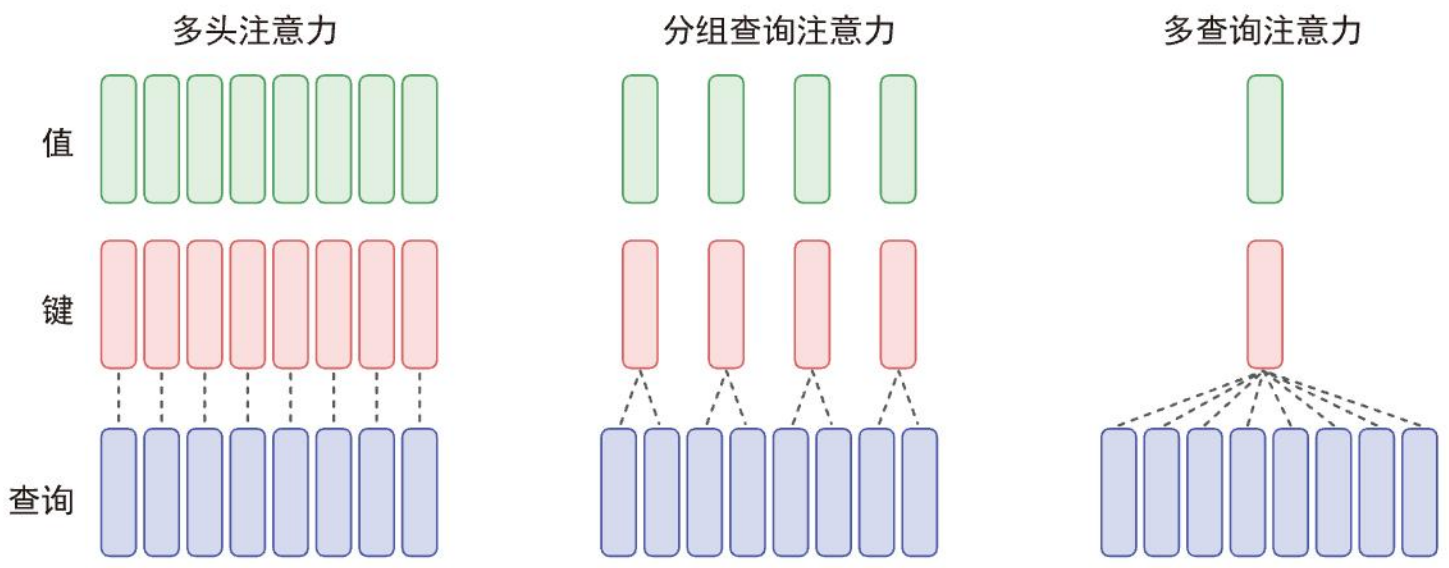


图 4.3　GQA 的 KV 头数介于 MHA 和 MQA 之间

MHA

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 各维护 h 个独立的头。

MQA

所有查询头共享 1 个键头和 1 个值头，$\boldsymbol{W}_k, \boldsymbol{W}_v$ 的参数规模显著减小。

GQA

把 h 个查询头分成 g 组，每组共享 1 个键头与 1 个值头，共 g 个 KV 头。

Llama-2 70B 与 Llama-3 系列均采用 GQA：减少参数量的同时，效果接近标准多头注意力。

GQA 的分组规则

设查询头总数为 h，分成 g 组，须满足 g 整除 h、$1 < g < h$；每组查询头数量为 $h' = h/g$。

$$h' = \frac{h}{g}, \qquad \boldsymbol{Q}^{(1..h')}, \boldsymbol{Q}^{(h'+1..2h')}, \cdots \text{ 共享同一组 } \boldsymbol{K}, \boldsymbol{V}$$

每个 KV 头被组内的 h' 个查询头共享；计算注意力时，该组的共享 KV 头与组内全部 h' 个查询头做注意力。

参数

投影矩阵变窄

$\boldsymbol{W}_k, \boldsymbol{W}_v$ 的维度变为 $\mathbb{R}^{d\times\left(g\cdot\frac{d}{h}\right)}$；MHA 中为 $\mathbb{R}^{d\times d}$。

计算量

注意力计算并未减少

投影参数减少，但注意力特征的计算量与 MHA 相同——每个查询头仍要做完整的注意力。

例：$h = 32$、$g = 8$ 时，每组 $h' = 4$ 个查询头共享 1 个键头与 1 个值头，$\boldsymbol{W}_k, \boldsymbol{W}_v$ 的输出维度降为 $d/4$。

GQA 的计算流程

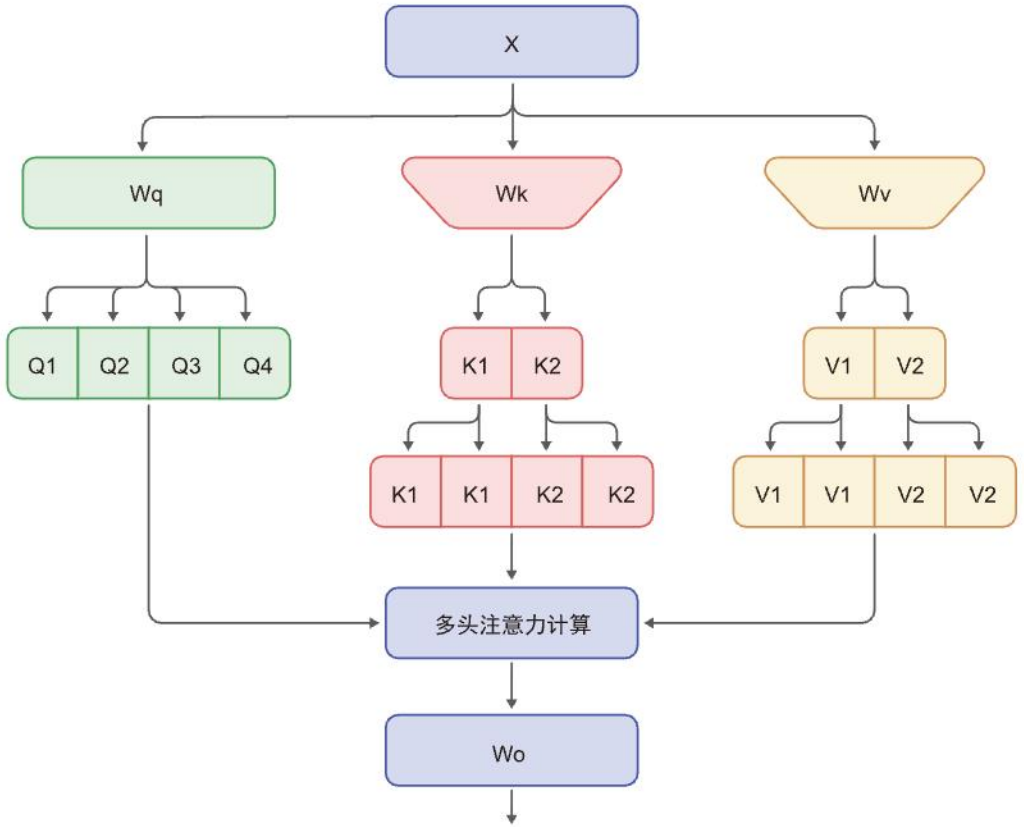


图 4.4　GQA 计算流程图

- 输入 $\boldsymbol{X}$ 经 $\boldsymbol{W}_q$ 得到 h 个查询头，经 $\boldsymbol{W}_k, \boldsymbol{W}_v$ 只得到 g 个键头与值头。
- 组内的 h' 个查询头与同一组 $\boldsymbol{K}, \boldsymbol{V}$ 计算注意力，结果拼接后经 $\boldsymbol{W}_o$ 输出。
- 推理时只需缓存 g 个 KV 头（而非 h 个），KV-Cache 空间显著下降，可用更大批次解码，提升吞吐。

Llama 同时移除了 $\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v, \boldsymbol{W}_o$ 的偏置项：减少参数，并避免学习与预训练任务无关的固定偏移。

图 《大语言模型：训练、精调与对齐》图 4.4

绝对位置编码的问题

Transformer 在输入层把位置向量 $\boldsymbol{p}_m$ 加到词嵌入 $\boldsymbol{x}_m$ 上，再投影为查询与键（忽略偏置）：

$$\boldsymbol{q}_m = (\boldsymbol{x}_m + \boldsymbol{p}_m)\boldsymbol{W}_q \tag{4-28}$$

$$\boldsymbol{k}_n = (\boldsymbol{x}_n + \boldsymbol{p}_n)\boldsymbol{W}_k \tag{4-29}$$

$$\begin{aligned}\boldsymbol{q}_m\boldsymbol{k}_n^T &= (\boldsymbol{x}_m\boldsymbol{W}_q + \boldsymbol{p}_m\boldsymbol{W}_q)(\boldsymbol{x}_n\boldsymbol{W}_k + \boldsymbol{p}_n\boldsymbol{W}_k)^T \\ &= \boldsymbol{x}_m\boldsymbol{W}_q\boldsymbol{W}_k^T\boldsymbol{x}_n^T + \boldsymbol{x}_m\boldsymbol{W}_q\boldsymbol{W}_k^T\boldsymbol{p}_n^T \\ &\quad + \boldsymbol{p}_m\boldsymbol{W}_q\boldsymbol{W}_k^T\boldsymbol{x}_n^T + \boldsymbol{p}_m\boldsymbol{W}_q\boldsymbol{W}_k^T\boldsymbol{p}_n^T\end{aligned} \tag{4-30/31}$$

四项依次为：内容–内容、内容–位置、位置–内容、位置–位置。

- 只有 $\boldsymbol{p}_m\boldsymbol{W}_q\boldsymbol{W}_k^T\boldsymbol{p}_n^T$ 仅与位置相关，能表征相对位置关系。
- $\boldsymbol{x}_m\boldsymbol{W}_q\boldsymbol{W}_k^T\boldsymbol{p}_n^T$ 与 $\boldsymbol{p}_m\boldsymbol{W}_q\boldsymbol{W}_k^T\boldsymbol{x}_n^T$ 让注意力表征受绝对位置影响：相对位置相同，绝对位置不同，输出就不同。
- 例：序列 (大, 模, 型) 与 (学, 习, 大, 模, 型) 中，词元对 (大, 型) 的分数不同，$\boldsymbol{W}_q, \boldsymbol{W}_k$ 因而学到绝对位置偏置。

旋转位置编码（RoPE）

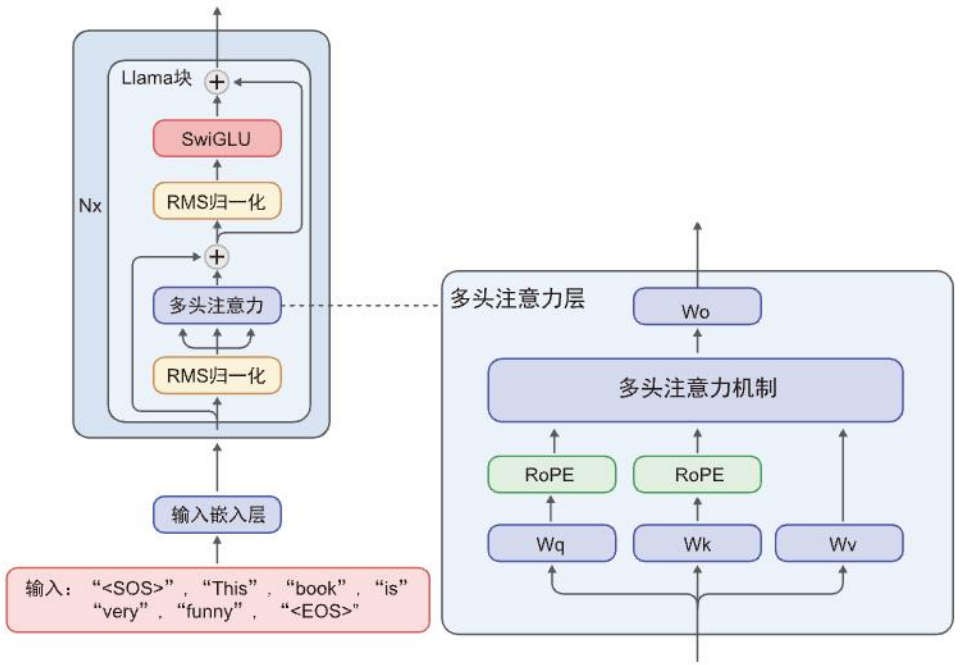


图 4.5　旋转位置编码作用在注意力计算之前嵌入

- RoFormer 提出的 RoPE 不在输入层加位置向量，而是在多头注意力计算前，对 $\boldsymbol{q}$、$\boldsymbol{k}$ 施加旋转。
- 注意力分数因此只体现向量之间的纯相对位置信息，与绝对位置无关。
- RoPE 作用于每一个注意力模块：位置信息不会像绝对位置编码那样随网络深度增加而衰减。
- 旋转位置编码是不可学习参数，可按最大位置 n 与单头维度 d 预计算并缓存 $\sin(n\theta_i)$、$\cos(n\theta_i)$。

RoPE 的二维旋转

定义角度为 θ 的二维旋转矩阵 $\boldsymbol{R}(\theta) \in \mathbb{R}^{2\times 2}$；二维单位向量 $\boldsymbol{a} \in \mathbb{R}^2$ 绕原点旋转 θ 得到 $\boldsymbol{b}$。

$$\boldsymbol{b} = \boldsymbol{a}\boldsymbol{R}(\theta) = \boldsymbol{a}\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \tag{4-32}$$

行向量右乘矩阵：结果的第 1 个分量为 $a_1\cos\theta + a_2\sin\theta$，第 2 个分量为 $-a_1\sin\theta + a_2\cos\theta$。

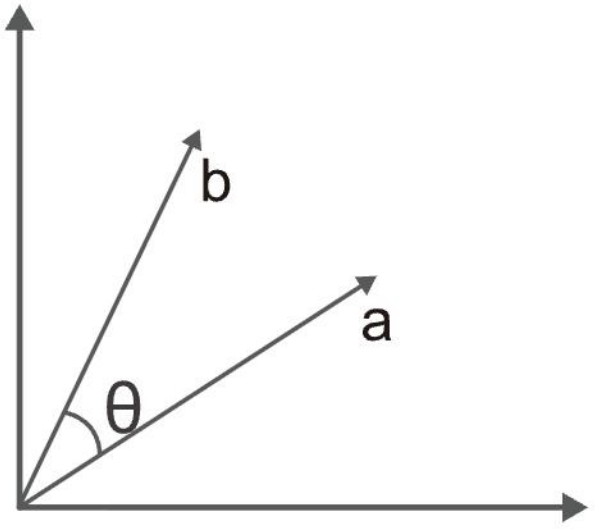


图 4.6　二维向量旋转示意图

$$\boldsymbol{R}(m,\theta) = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \tag{4-33}$$

给定位置 m 与旋转角度 θ，位置 m 对应的旋转矩阵记为 $\boldsymbol{R}(m,\theta)$，简写为 $\boldsymbol{R}_m$。

RoPE 推导（一）

位置 m、n 处的二维向量 $\boldsymbol{q}_m, \boldsymbol{k}_n$ 各自旋转后求点积，得到

$$(\boldsymbol{q}_m \boldsymbol{R}_m)(\boldsymbol{k}_n \boldsymbol{R}_n)^T = \boldsymbol{q}_m \boldsymbol{R}_m \boldsymbol{R}_n^T \boldsymbol{k}_n^T \tag{4-34}$$

θ 固定，$\boldsymbol{R}(m,\theta)$、$\boldsymbol{R}(n,\theta)$ 简写为 $\boldsymbol{R}_m$、$\boldsymbol{R}_n$。关键是中间的 $\boldsymbol{R}_m \boldsymbol{R}_n^T$。

$$\boldsymbol{R}_m \boldsymbol{R}_n^T = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} \cos n\theta & \sin n\theta \\ -\sin n\theta & \cos n\theta \end{bmatrix} \tag{4-35}$$

$$= \begin{bmatrix} \cos m\theta \cos n\theta + \sin m\theta \sin n\theta & \cos m\theta \sin n\theta - \sin m\theta \cos n\theta \\ \sin m\theta \cos n\theta - \cos m\theta \sin n\theta & \sin m\theta \sin n\theta + \cos m\theta \cos n\theta \end{bmatrix} \tag{4-36}$$

$$= \begin{bmatrix} \cos(m-n)\theta & -\sin(m-n)\theta \\ \sin(m-n)\theta & \cos(m-n)\theta \end{bmatrix} = \boldsymbol{R}_{m-n} \tag{4-37}$$

式 (4-35)：$\boldsymbol{R}_n^T$ 中 sin 的符号翻转；式 (4-36) 逐元素相乘并相加；式 (4-37) 用和差角公式合并，认出这正是角度为 $(m-n)\theta$ 的旋转矩阵，即式 (4-38)。

RoPE 推导（二）

$$(\boldsymbol{q}_m\boldsymbol{R}_m)(\boldsymbol{k}_n\boldsymbol{R}_n)^T = \boldsymbol{q}_m\boldsymbol{R}_m\boldsymbol{R}_n^T\boldsymbol{k}_n^T = \boldsymbol{q}_m\boldsymbol{R}_{m-n}\boldsymbol{k}_n^T \tag{4-39}$$

把式 (4-38) 代入式 (4-34)：两个经旋转的向量做点积，结果依赖相对位置差 $m-n$。

从二维扩展到 d 维：每两个维度为一组，共 $d/2$ 组，旋转矩阵是 $d/2$ 个二维旋转矩阵沿对角线的排列。

$$\boldsymbol{R}_m^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \tag{4-40}$$

$\theta_i = 10\,000^{-2(i-1)/d}$，$i = 1, 2, \cdots, d/2$。维度越低（$i$ 越小），θ_i 越大，旋转越快。

标量视角　式 (4-39) 的结果是一个**标量**。按 $d/2$ 组展开，第 i 组贡献 $a_i\cos(t\theta_i) + b_i\sin(t\theta_i)$（$t = m-n$，$a_i, b_i$ 只与内容有关）：中间点积的每一项都被一个 **由相对位置 t 与该组维度 i 共同决定的系数**加权，再求和——即"与相对位置/维度有关的缩放加权求和"。

RoPE 推导（三）

先把位置 m、n 的 d 维词元特征投影为 $\boldsymbol{q}_m = \boldsymbol{x}_m \boldsymbol{W}_q$、$\boldsymbol{k}_n = \boldsymbol{x}_n \boldsymbol{W}_k$，再在计算注意力分数前施加 RoPE：

$$\left(\boldsymbol{q}_m \boldsymbol{R}_m^d\right)\left(\boldsymbol{k}_n \boldsymbol{R}_n^d\right)^T = \boldsymbol{q}_m \boldsymbol{R}_{m-n}^d \boldsymbol{k}_n^T = \boldsymbol{x}_m \boldsymbol{W}_q \boldsymbol{R}_{m-n}^d \boldsymbol{W}_k^T \boldsymbol{x}_n^T \tag{4-41}$$

$\boldsymbol{q}_m, \boldsymbol{k}_n \in \mathbb{R}^d$。旋转矩阵夹在 $\boldsymbol{W}_q$ 与 $\boldsymbol{W}_k^T$ 之间，只贡献相对位置 $m-n$。

- 经 RoPE 变换后的注意力分数只含相对位置关系，不受绝对位置影响。
- 学习过程中，$\boldsymbol{W}_q$、$\boldsymbol{W}_k$ 会适应预设的旋转矩阵参数分布。
- RoPE 只作用于查询与键，不作用于值向量：值向量还要参与注意力权重的加权组合，对其旋转会破坏权重的纯相对位置表示。

对照 式 (4-31) 的四个交叉项在 RoPE 下被旋转矩阵整体吸收，最终只剩一个依赖 $m-n$ 的旋转矩阵。

RoPE 的角度设置

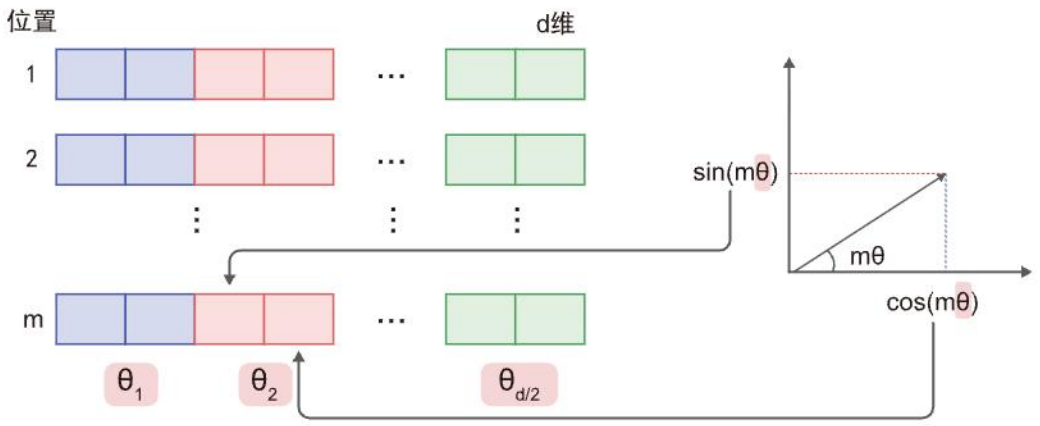


图 4.7　RoPE 旋转位置编码示意图

- 旋转角度由位置 m 与维度 i 共同决定：$\theta_i = b^{-2(i-1)/d}$，其中 $b = 10\,000$。
- d 维向量元素两两成组，共 $d/2$ 组，每组用自己的 θ_i。
- 按位置 m 算出 $\cos(m\theta_i)$、$\sin(m\theta_i)$，即可用线性化公式对词向量做变换。
- 每组角度不同，使得不同维度对位置的分辨率不同：低维旋转快，高维旋转慢（4.4.3 展开）。

RoPE 的高效实现

$\boldsymbol{R}_m^d$ 是稀疏的伪对角矩阵，直接做矩阵乘法效率低。利用分组结构，可等价写成“逐元素乘 + 加法”：

$$\boldsymbol{R}_m^d \boldsymbol{x}^T = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_d \\ x_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix} \tag{4-42}$$

$\otimes$ 为逐元素乘积。第 1、2 维：$x_1 \cos m\theta_1 - x_2 \sin m\theta_1$ 与 $x_1 \sin m\theta_1 + x_2 \cos m\theta_1$，正是二维旋转。

- RoPE 无学习参数：初始化时按最大位置 n、单头维度 d 预计算所有位置的 sin、cos 分量。
- 应用时按位置 m 查表，按式 (4-42) 做逐元素乘与加，避免构造 $d \times d$ 稀疏矩阵。
- 不使用位置编码的 NoPE 在长度泛化任务中优于所有显式位置编码方法，且没有额外计算开销。

依据 《大语言模型：训练、精调与对齐》第 4.2.5 节（式 (4-42)）

为什么不对 v 施加 RoPE

RoPE 只作用在 q、k 上。值向量 v 的任务不是"被比较"，而是"按权重被搬运"，旋转它会改变搬运出去的内容本身。

理由一 · 位置只需进入比较

注意力分数 $q_m R_{m-n} k_n^T$ 已经携带相对位置；输出 $\sum_n \alpha_{mn} v_n$ 是内容加权和，位置信息通过 α 进入即可。

理由二 · 旋转不消去

对 v_n 旋转后，输出变成 $\sum_n \alpha_{mn} v_n R_n$，**各位置带上不同的旋转矩阵**，加权和不再落在同一个基上，无法化简为相对位置。

理由三 · 破坏残差与缓存

v 直接进入残差主干与 KV-Cache；加旋转等于给主干注入随位置变化的坐标变换，并让缓存无法跨位置复用。

反例 · VO-RoPE

已有工作（VO-RoPE）确实把旋转加到值向量上，做法是让 v 的旋转与输出侧的解旋转配对，使相对位置在输出端仍可对齐——代价是多一次还原运算。

结论 qk 的注意力分数 $q_m R_{m-n} k_n^T$ 是一个嵌入纯相对距离的表示；若对 v 施加 RoPE 再做注意力加权求和，输出 $\sum_n \alpha_{mn} v_n R_n$ 里各位置带着以绝对位置为角度的旋转，**相对距离表示随之被破坏**。

依据 《大语言模型：训练、精调与对齐》第 4.2.5 节；VO-RoPE 为后续工作

SwiGLU 前馈层

$$\begin{aligned}\mathtt{SwiGLU}(\boldsymbol{x},\boldsymbol{W},\boldsymbol{V},\boldsymbol{W}_2) &= (\mathtt{Swish}(\boldsymbol{x}\boldsymbol{W})\otimes \boldsymbol{x}\boldsymbol{V})\,\boldsymbol{W}_2\\ \mathtt{Swish}(\boldsymbol{x}) &= \boldsymbol{x}\,\sigma(\boldsymbol{x})\end{aligned}$$

$\boldsymbol{W},\boldsymbol{V}\in\mathbb{R}^{d\times\frac{8}{3}d}$，$\boldsymbol{W}_2\in\mathbb{R}^{\frac{8}{3}d\times d}$；$\sigma$ 为 sigmoid；$\otimes$ 为逐元素乘积；所有线性层无偏置。

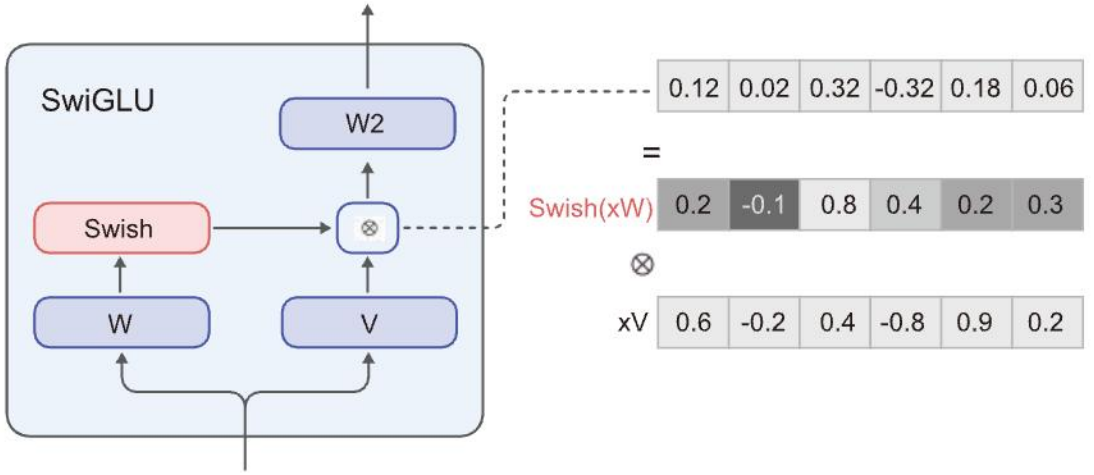


图 4.8　SwiGLU 计算图

$$3\times d\times\frac{8}{3}d=8d^2 \qquad \text{对比} \qquad 2\times d\times 4d=8d^2$$

$\boldsymbol{W}$、$\boldsymbol{V}$、$\boldsymbol{W}_2$ 的参数量合计 $8d^2$；常规前馈层 $\boldsymbol{W}_1\in\mathbb{R}^{d\times 4d}$、$\boldsymbol{W}_2\in\mathbb{R}^{4d\times d}$ 也是 $8d^2$。把投影维度取 $\frac{8}{3}d$，正是为了与标准前馈层参数量持平。

Swish 激活函数

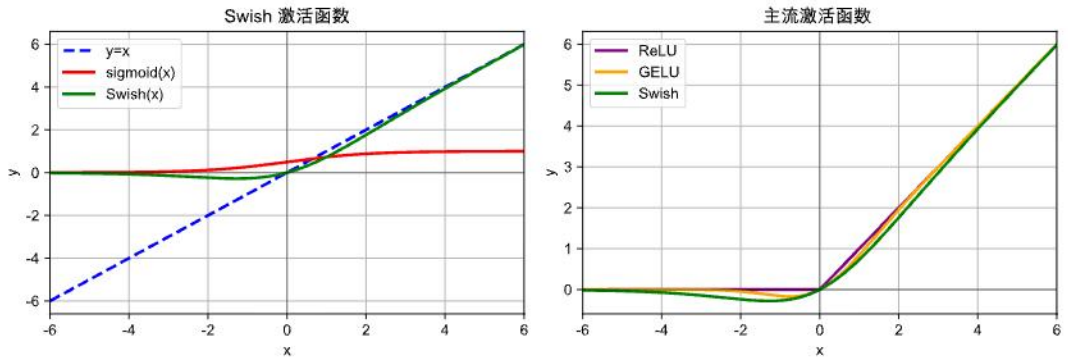


图 4.9　Swish 激活函数与主流激活函数

$$y(x) = x \cdot \mathtt{Sigmoid}(x)$$

$$y(x) = x\,\sigma(\alpha x) \quad \text{（通用形式）}$$

$\alpha = 1$ 时两式相同；α 作为可学习参数时，门控的陡峭程度由数据决定。

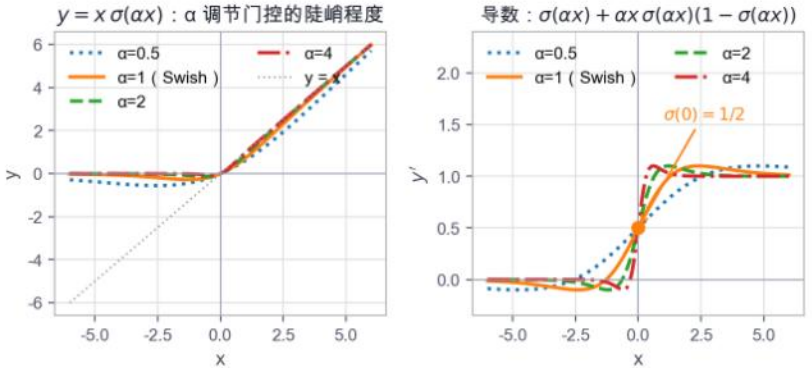


图 4.9b　$x\,\sigma(\alpha x)$ 及导数随 α 的变化

- 比 ReLU 更平滑，且在 0 点可导，避免 ReLU 在负输入区的梯度消失。
- 非线性特性增强特征表达；与 GELU 相比，Swish 的计算更简便。
- **求导简便**：$y' = \sigma(x) + x\sigma(x)(1 - \sigma(x))$，用前向已算出的 $\sigma(x)$ 即可，无需额外的特殊函数。
- α 调节门控陡峭程度：$\alpha \to \infty$ 趋近 ReLU，$\alpha \to 0$ 趋近线性。

SwiGLU 的两条分支

门控机制通过可学习权重，按输入自适应调节各特征的重要性。SwiGLU 的两条投影分支正好对应特征学习与特征选择。

分支 xV · 特征学习

把输入映射到更高维

投影参数 V 将 x 映射为高维表示 xV，增强表达能力；该表示再与门控信号交互，最后经 W_2 回到原维度。

分支 $\mathrm{Swish}(xW)$ · 特征选择

逐元素缩放，软选择

W 生成的门控信号对 xV 逐元素缩放，动态筛选传给下一层的特征。

权重可降低但不归零，保留有用信号。

- 门控是动态的：不同输入生成不同的门控信号，模型据此决定保留或过滤哪些特征通道。
- SwiGLU 提升了输入信息的利用效率，尤其适合高维、稀疏或含噪声的数据，已成为大模型提升表征能力的关键组件。

两种“特征选择”：归一化与门控

γ 与 SwiGLU 门控都在给特征分配权重，但作用范围不同：前者面向所有词向量共用一套，后者是每个词元独有的。

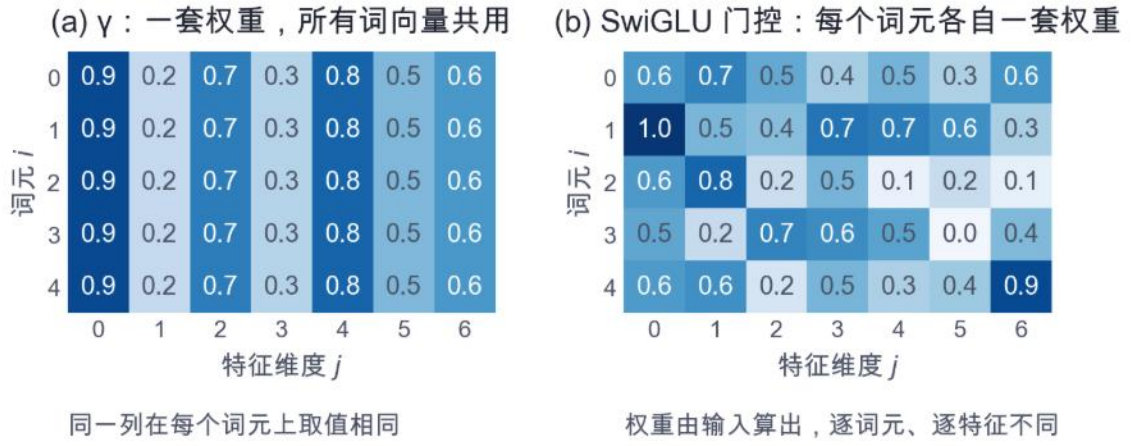


图 4.8b　γ 与 SwiGLU 门控的权重分布

归一化的 γ：与词元无关

是一组固定维度上的可学习参数，对同一层内所有词向量施加**同一套**缩放宽重；它不读取输入内容，因此不做“选择”。

SwiGLU 门控：逐词元、逐特征

门控信号 $\mathtt{Swish}(\boldsymbol{x}\boldsymbol{W})$ 由当前词元算出，每个位置、每个特征各有一个权重，按内容决定保留多少。

两者相乘时并不冲突：归一化先统一尺度、让 γ 学一层全局的通道重要性；门控再按当前输入做一次实例级的筛选。前者是**静态的、面向全部词向量的**，后者是**动态的、各特征独有的**——这正是 SwiGLU 带来的一类新特征选择模式。

4.3

键值缓存

本节内容

4.3.1 自回归模型生成分析

4.3.2 KV-Cache 的实现

4.3.3 KV-Cache 显存消耗

自回归生成每次只新增一个词元，却反复重算整段历史。KV-Cache 用空间换时间：把已经算过的键和值留下来。

逐词元生成的计算过程

解码器模型推理时，需要输入词元序列并逐个词元生成。长度为 $t-1$ 的输入 $\boldsymbol{x}_{1:t-1}$ 前向计算得到 x_t，新词元再用于下一轮得到 x_{t+1}。

$$f_\theta(\boldsymbol{x}_{1:t-1}) = \arg\max_{v \in \mathcal{V}} p_\theta(x_v \mid \boldsymbol{x}_{1:t-1}) \tag{4-44}$$

贪婪解码：在词表 $\mathcal{V}$ 中取概率最大的词元。$p_\theta(x_v \mid \boldsymbol{x}_{1:t-1})$ 是模型对下一个词元 v 的预测分布。



生成 T 个词元就要做 T 次前向，而每次的输入只比上一次多一个词元——这正是重复计算的来源。

依据 《大语言模型：训练、精调与对齐》第 4.3.1 节（式 (4-44)）

无缓存时的重复计算

输入 $\boldsymbol{x}_{1:t}$ 经嵌入层得到 $\boldsymbol{X}_{1:t}$，再投影并计算注意力（$\mathtt{Attn}(\cdot)$ 为缩放点积注意力）：

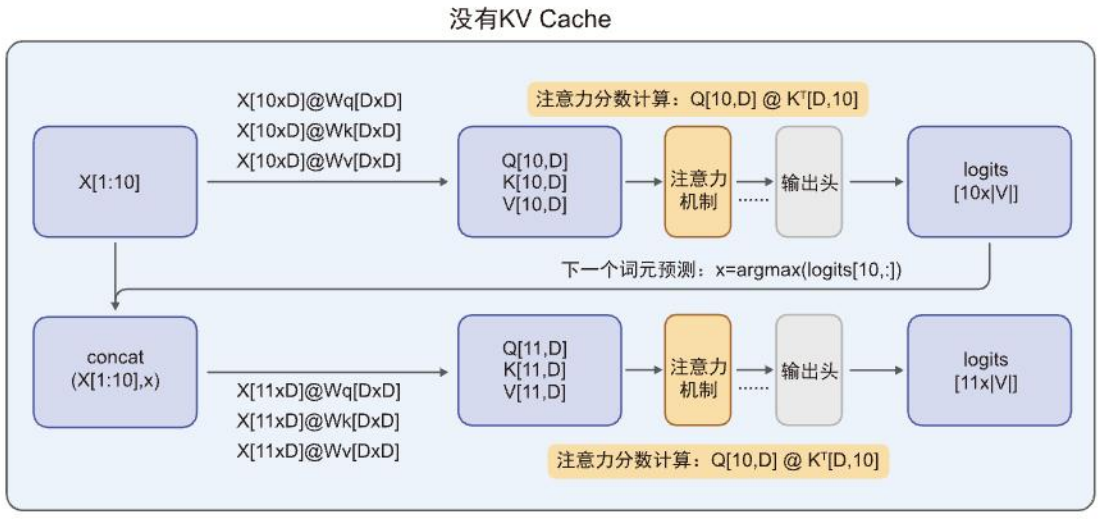
$$\boldsymbol{Q}_{1:t} = \boldsymbol{X}_{1:t}\boldsymbol{W}_q \tag{4-45}$$
$$\boldsymbol{K}_{1:t} = \boldsymbol{X}_{1:t}\boldsymbol{W}_k \tag{4-46}$$
$$\boldsymbol{V}_{1:t} = \boldsymbol{X}_{1:t}\boldsymbol{W}_v \tag{4-47}$$
$$\boldsymbol{Z}_{1:t} = \mathtt{Attn}(\boldsymbol{Q}_{1:t}, \boldsymbol{K}_{1:t}, \boldsymbol{V}_{1:t}) \tag{4-48}$$

$\boldsymbol{Z}_{1:t} \in \mathbb{R}^{t\times d}$：一次并行得到 t 个位置的特征，但自回归生成只需要最后一个位置。

$$\boldsymbol{Q}_{1:t+1} = \boldsymbol{X}_{1:t+1}\boldsymbol{W}_q \tag{4-49}$$
$$\boldsymbol{K}_{1:t+1} = \boldsymbol{X}_{1:t+1}\boldsymbol{W}_k \tag{4-50}$$
$$\boldsymbol{V}_{1:t+1} = \boldsymbol{X}_{1:t+1}\boldsymbol{W}_v \tag{4-51}$$
$$\boldsymbol{Z}_{1:t+1} = \mathtt{Attn}(\boldsymbol{Q}_{1:t+1}, \boldsymbol{K}_{1:t+1}, \boldsymbol{V}_{1:t+1}) \tag{4-52}$$

$\boldsymbol{x}_{:t+1} \leftarrow (\boldsymbol{x}_{:t}, x_{t+1})$：新词元拼到末尾，但 $\boldsymbol{X}_{1:t}\boldsymbol{W}_k$、$\boldsymbol{X}_{1:t}\boldsymbol{W}_v$ 又被算了一遍。

重复计算的位置



- 处理 $\boldsymbol{x}_{1:t}$ 时并行算了 t 个位置的预测，实际只要最后一个位置。
- $t+1$ 时刻重复计算了 t 时刻已得到的 $\boldsymbol{K}_{1:t}$、$\boldsymbol{V}_{1:t}$——这两项可以缓存。
- $t+1$ 时刻还算了多个查询向量的注意力输出，实际只要第 $t+1$ 个位置。

图 4.10　无 KV-Cache 时，推理时注意力存在大量的重复计算

图中的 $\boldsymbol{X}[10\times D]@\boldsymbol{W}_k[D\times D]$ 与 $\boldsymbol{X}[11\times D]@\boldsymbol{W}_k[D\times D]$ 只差最后一行，前 10 行完全相同。

键值缓存的计算方式

首次计算时把各注意力模块生成的 $\boldsymbol{K}$、$\boldsymbol{V}$ 存为 KV-Cache；之后每步只需处理最新词元 x_{t+1}。

$$\boldsymbol{Q}_{t+1} = \boldsymbol{X}_{t+1}\boldsymbol{W}_q \tag{4-53}$$
$$\boldsymbol{K}_{t+1} = \boldsymbol{X}_{t+1}\boldsymbol{W}_k \tag{4-54}$$
$$\boldsymbol{V}_{t+1} = \boldsymbol{X}_{t+1}\boldsymbol{W}_v \tag{4-55}$$
$$\boldsymbol{K}_{1:t+1} \leftarrow \texttt{Concat}(\boldsymbol{K}_{1:t}, \boldsymbol{K}_{t+1}) \tag{4-56}$$
$$\boldsymbol{V}_{1:t+1} \leftarrow \texttt{Concat}(\boldsymbol{V}_{1:t}, \boldsymbol{V}_{t+1}) \tag{4-57}$$
$$\boldsymbol{Z}_{t+1} = \texttt{Attn}(\boldsymbol{Q}_{t+1}, \boldsymbol{K}_{1:t+1}, \boldsymbol{V}_{1:t+1}) \tag{4-58}$$

$\boldsymbol{Q}_{t+1}, \boldsymbol{K}_{t+1}, \boldsymbol{V}_{t+1} \in \mathbb{R}^{1\times d}$。Concat 沿序列维度拼接缓存与当前步；$\boldsymbol{Z}_{t+1}$ 只有一个位置。

- 只需单个查询向量 $\boldsymbol{Q}_{t+1}$ 与完整键值序列 $\boldsymbol{K}_{1:t+1}$、$\boldsymbol{V}_{1:t+1}$ 计算注意力特征。
- 历史序列的投影变换不再重算，注意力输出也不再为 t 个位置重复计算。

依据 《大语言模型：训练、精调与对齐》第 4.3.1 节（式 (4-49)–(4-52)）

预填充与解码两个阶段

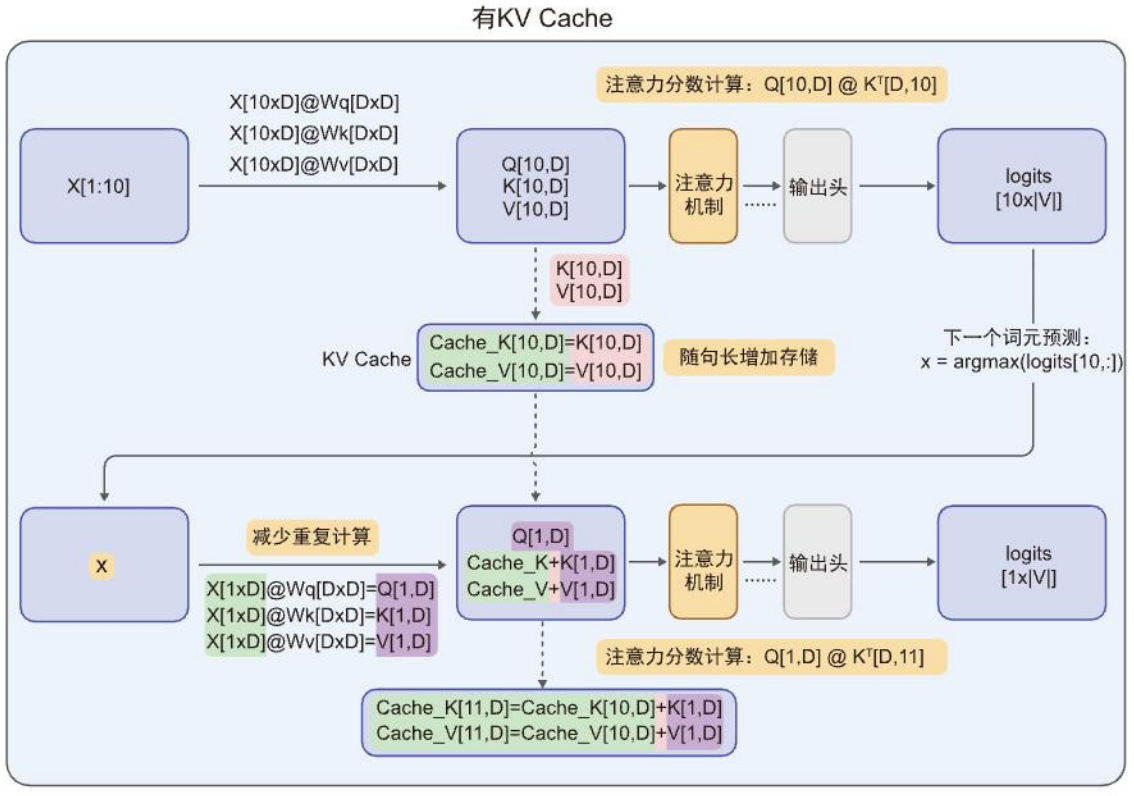


图 4.11　有 KV-Cache 时，生成过程可分为预填充和解码阶段

预填充（Prefill）

首次处理完整输入序列，在每层注意力中计算并缓存 $\boldsymbol{K}$、$\boldsymbol{V}$。

解码（Decoding）

每步只处理 1 个新词元，与缓存键值对一起算注意力，输出下一个词元。

缓存随句长增加而增长；换来的是解码阶段不再重复历史投影。

键值缓存的复杂度

KV-Cache 是注意力类模型的通用推理加速方法，适用于所有含解码器的 Transformer，并非 Llama 首创。

不使用 KV-Cache

每步都重算长度 t 的序列投影 $\boldsymbol{X}_{:t}\boldsymbol{W}$。

每步都要算 t 个位置的注意力输出。

时间复杂度随序列长度增加而显著增加。

使用 KV-Cache

解码阶段只算新词元的投影 $\boldsymbol{X}_{t+1}\boldsymbol{W}$。

只算单个查询向量 $\boldsymbol{Q}_{t+1}$ 的注意力输出。

计算量大幅减少，代价是额外的 KV-Cache 存储。

> **权衡** 用空间换时间。生成长序列时，这一权衡通常能大幅提升推理效率；而缓存本身的显存开销，就是 4.3.3 要解决的问题。

生成循环

预填充阶段需要完整序列 $\boldsymbol{x}_{1:t}$ 预测下一个词元；解码阶段有缓存，每轮输入 x_t 只有 1 个词元。

```
x = torch.randint(low=1, high=vocab_size, size=(1, seq_len))
mask = torch.triu(torch.ones(1, 1,
                             seq_len+max_new_tokens,
                             seq_len+max_new_tokens,
                             dtype=torch.bool))[:, :, 1:, :-1]
pos = torch.arange(seq_len)

# 带 KV-Cache 的模型
model = DecodingModelWithKVCache(dim, heads, vocab_size)
for i in range(max_new_tokens):
    logits = model(x, mask[:, :, pos, pos])
    next_tokens = torch.argmax(logits, dim=-1)

    # x = torch.cat((x, next_tokens[:,[-1]]), dim = -1)
    x = next_tokens[:, [-1]]  # 有 KV-Cache 更新情况

    pos = torch.tensor([seq_len+i])
    if next_tokens[0, -1] == EOS_token_ID:
        break
```

与无缓存版本的关键差别在注释一行：不再把新词元拼回输入，而是只把最新词元送入模型，历史由缓存承担。

依据 《大语言模型：训练、精调与对齐》第 4.3.2 节（代码 4）

注意力层的初始化与投影

```
class MultiHeadAttentionWithKVCache(nn.Module):
    def __init__(self, dim, heads):
        super(MultiHeadAttentionWithKVCache, self).__init__()
        self.heads = heads
        self.dim = dim
        self.WQ = nn.Linear(dim, dim)
        self.WK = nn.Linear(dim, dim)
        self.WV = nn.Linear(dim, dim)
        self.WO = nn.Linear(dim, dim)
        self.cache_K = None
        self.cache_V = None

    def forward(self, X, mask):
        B, L, D = X.size()
        head_dim = self.dim // self.heads  # 单头词向量维度
        Q, K, V = self.WQ(X), self.WK(X), self.WV(X)
        Q = Q.reshape(B, L, self.heads, head_dim).transpose(1, 2)
        K = K.reshape(B, L, self.heads, head_dim).transpose(1, 2)
```

- 四个线性层都不带偏置，与 4.2.1 的取舍一致；`cache_K`、`cache_V` 初始为 `None`。
- `head_dim = dim // heads` 即单头维度 d'；Q, K, V 被 reshape 成 `(B, heads, L, head_dim)`。
- 投影只作用于本步输入 X：解码阶段 $L = 1$，投影开销与序列长度无关。

缓存拼接与注意力计算

```
if self.cache_K is None:
    self.cache_K = K
    self.cache_V = V
else:
    self.cache_K = torch.cat((self.cache_K, K), dim=2)
    self.cache_V = torch.cat((self.cache_V, V), dim=2)

Z = ScaledDotProductAttention(Q,
                              self.cache_K, self.cache_V,
                              mask)
Z = Z.transpose(1, 2).reshape(B, L, self.dim)
O = self.WO(Z)
return O
```

- 首次调用对应预填充：直接把本步的 K、V 赋给缓存。
- 之后沿序列维 `dim=2` 拼接：缓存始终是“历史 + 当前步”的完整 $\boldsymbol{K}$、$\boldsymbol{V}$。
- 注意力的查询只来自本步输入 Q（解码时 $L=1$），键与值来自完整缓存。

依据 《大语言模型：训练、精调与对齐》第 4.3.2 节（代码 5）

输出投影

```
class DecodingModelWithKVCache(nn.Module):
    def __init__(self, dim, heads, vocab_size):
        super(DecodingModelWithKVCache, self).__init__()
        self.embd = nn.Embedding(vocab_size, dim)  # 忽略位置编码
        self.decoder = MultiHeadAttentionWithKVCache(dim, heads)  # 简化解码器
        self.norm = nn.LayerNorm(dim)
        self.lm_heads = nn.Linear(dim, vocab_size)

    def forward(self, x, mask):
        X = self.embd(x)
        X = self.decoder(X, mask)
        H = self.norm(X)
        logits = self.lm_heads(H)
        return logits
```

- 缓存管理加在注意力模块内部：预填充存储，解码时合并当前步的单个 KV 再计算注意力。
- 示例模型把解码器简化为单层注意力，位置编码由调用方以 pos 索引掩码的方式处理。

依据 《大语言模型：训练、精调与对齐》第 4.3.2 节（代码 5）

键值缓存的大小

KV-Cache 用空间换时间。解码阶段，缓存要同时保存键与值，其存储量为：

$$\Psi = l \times t \times d' \times h' \times k_{\mathrm{b}} \times 2 \tag{4-59}$$

l 为注意力层数，t 为序列长度，$d' = d/h$ 为每个注意力头的维度，h' 为需要缓存的键值头数（MHA 下 $h' = h$，GQA 下 $h' = g$），k_{b} 为存储数值类型的位数，末尾的 2 表示键与值各存一份。

$\propto t$

随序列长度线性增长

$\propto l$

每层各存一份

$\times 2$

键与值

长序列下缓存迅速膨胀，容易导致显存不足——这正是下一组优化方向的出发点。

减小缓存的五个方向

$\Psi = l \times t \times d' \times h' \times k_{\mathrm{b}} \times 2$ 的每个因子，都对应一条优化路径。

缩短 t

减少序列长度

窗口注意力限制计算范围；或用稀疏注意力减少键值对数量。

降低 l

层间缓存共享

相邻层共用 KV-Cache，降低层数因子的影响。

减少 h'

优化注意力头

用 GQA 或 MQA 减少需要存储的键值头数量。

压缩 d'

降低表示维度

学习低维潜在表示（$\tilde{d} \ll d$，$h = 1$）作为缓存，如 MLA。

第五条是压缩存储精度（k_{b}）：对 KV-Cache 量化。此外，分页注意力借鉴操作系统的分页机制提高存储利用率；部署时还可按预填充（计算密集）与解码（访存密集）分离算力。

GQA 的缓存大小

$$\Psi_t = l \times t \times d' \times h' \times k_{\mathrm{b}} \times 2 \tag{4-60}$$

h' 为分组后的键值头数量。MHA 需缓存 h 个头，GQA 只需 g 个，缓存降为原来的 g/h。

解码阶段：处理第 i 层注意力时，先算注意力分数，再把 h' 个键值头的数据在各自的查询组内复制，以匹配组内查询头数量。

存 h' 个 KV 头
缓存只保留组级状态

→

组内复制
匹配该组查询头数

→

算注意力分数
组内 h' 个查询头

→

仍只存 h' 个
计算后不扩大缓存

- 复制只发生在计算注意力分数之前，是临时的；计算完成后，KV-Cache 仍然仅存储 h' 个头的状态。
- 缓存变小意味着同样的显存可以容纳更大批次，批量解码的吞吐随之提升。

4.4

长文本继续预训练

上下文从 8192 扩到 128k，靠的不是加长输入，而是改位置编码并让模型重新适应。先看衰减为何失效，再看 PI 与 NTK-RoPE 如何修复。

本节内容

4.4.1 Llama-3 长文本训练方案

4.4.2 衰减特性分析*

4.4.3 旋转位置编码分析

4.4.4 位置编码内插

4.4.5 NTK-RoPE

4.4.6 长文本能力测评

长文本继续预训练

继续预训练（Continue Pre-Training，CPT）是在常规预训练之外，让模型掌握特定领域知识或能力的手段。

语言

特定语种文本

增强模型在该语言上的能力。

领域

代码与数学

提升相应领域的理解与生成能力。

垂直

专业知识

注入行业内容，掌握专业表述。

- 长上下文处理是理解复杂任务的基础：长篇小说、海量网页汇总、复杂文档查询、大型代码工程，都要求模型在超长上下文中辨识并利用关键信息。
- 受训练资源限制，预训练会预设最大上下文长度，输入文本按此裁剪，模型也只适应这一长度的数据分布。
- 扩展有效上下文长度的可行途径，是通过预训练或微调继续训练。

上下文长度的分阶段扩展

Llama-3 的上下文可达 128 000：在已完成预训练（基础长度 8192）的模型上分阶段抬高目标长度；DeepSeek-V3 则用更激进的 4k → 32k → 128k 两阶段。



- 每一阶段的目标长度翻倍，模型逐级适应新的位置分布，而不是一步跳到 128k。
- 为什么要改位置编码：原始 RoPE 在 16k–64k 的相对距离上，点积分数并未如预期有效衰减。办法是缩放底数 $b = 10\,000$（4.4.5）；位置编码一变，就必须继续训练让模型适应新的分布。

另一条路线 · DeepSeek-V3

4k → 32k → 128k：跨度更大的两阶段扩展

先用 32k 继续预训练约 1000 步，再用 128k 继续预训练约 1000 步；**位置编码的超参随阶段调整**，训练量远小于重新预训练。

长距离下的注意力分数

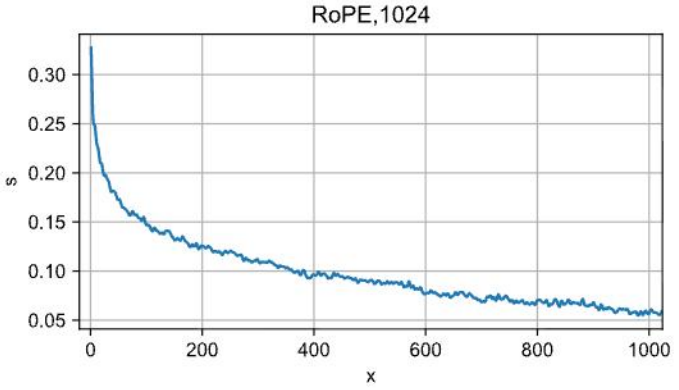


图 4.12(a)　上下文 1024：点积随相对距离稳步下降

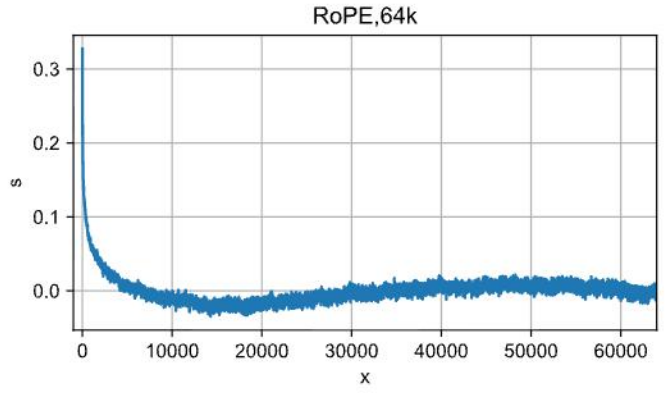


图 4.12(b)　上下文 64k：16k 之后点积停在 0 附近，不再衰减

$$\boldsymbol{q}_m \boldsymbol{R}_{m-n} \boldsymbol{k}_n^T$$

位置 m 与 n 的注意力分数；设 $\boldsymbol{q}_m, \boldsymbol{k}_n$ 为标准正态随机向量。

- 在 16k–64k 的相对距离上，点积分数没有按预期衰减。
- 距离越远、相关性本应越低；衰减失效意味着模型难以有效建模远距离依赖。
- 修法不是加长输入，而是改位置编码的表示方式，让长距离上稳定衰减、少震荡。

注意力分数的余弦形式

沿用 4.2.5 的 RoPE 形式，为分析随相对距离 $t = m - n$ 的衰减，取 $\boldsymbol{q}_m = \boldsymbol{k}_n = \mathbf{1} \in \mathbb{R}^d$（各元素独立同分布、均值 0、方差 1 的简化）：

$$f_t = \boldsymbol{q}_m \boldsymbol{R}_{m-n} \boldsymbol{k}_n^T \tag{4-61}$$

$$f_t = \mathbf{1}\, \boldsymbol{R}_t\, \mathbf{1}^T \tag{4-62}$$

$$= \sum_{i=1}^{d/2} \sin(m\theta_i)\sin(n\theta_i) + \cos(m\theta_i)\cos(n\theta_i) \tag{4-63}$$

$$= \sum_{i=1}^{d/2} \cos\left((m-n)\theta_i\right) \tag{4-64}$$

$$= \sum_{i=1}^{d/2} \cos\left(t\theta_i\right) \tag{4-65}$$

式 (4-63)：每组二维旋转贡献 $\sin\cdot\sin + \cos\cdot\cos$；式 (4-64)：用和差角公式合并为 $\cos((m-n)\theta_i)$，相对距离显式出现。

固定 $n = 0$ 时的衰减

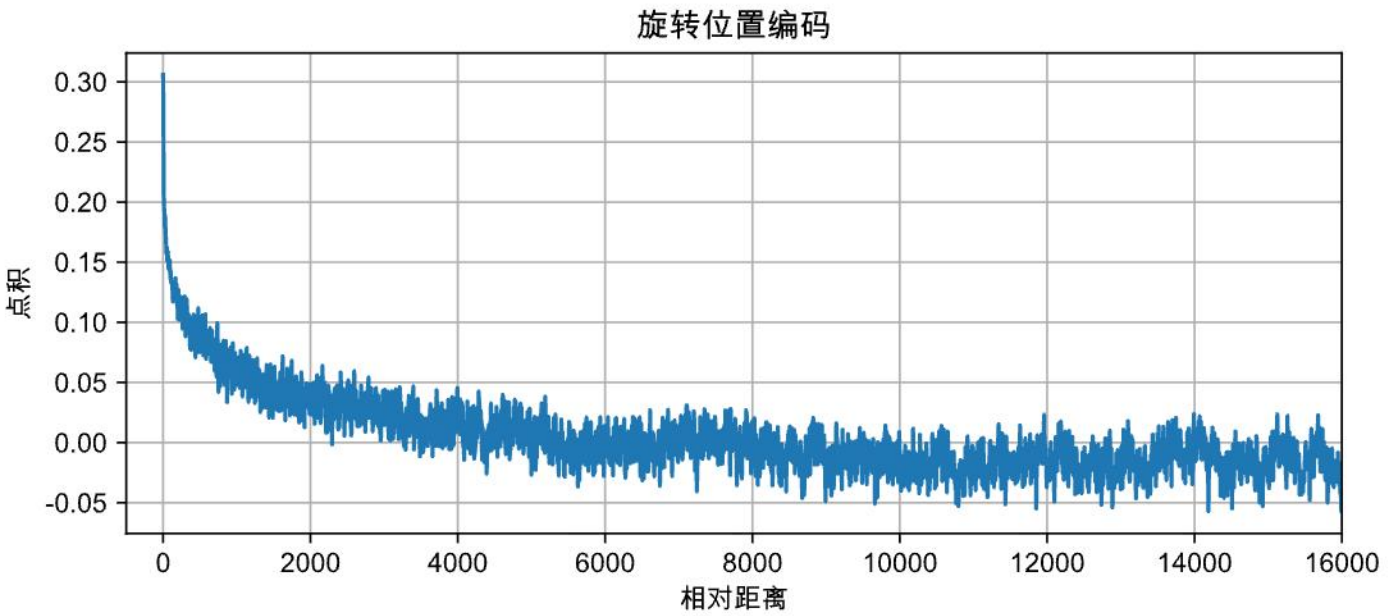


图 4.13　RoPE 位置编码点积衰减曲线图

固定键向量位置 $n = 0$，则 $t = m - 0 = m$，f_t 记为 f_m。

$$f_m = \sum_{i=1}^{d/2} \cos(m\theta_i) \quad (4\text{-}66)$$

$\theta_i = b^{-2(i-1)/d}$，共 $d/2$ 个不同频率的余弦叠加。

- 短相对距离内 f_m 明显衰减。
- 衰减不严格单调，在零附近震荡；f_m 可以为负。
- 包络的量级约为 $1/m$，下一节的证明给出 $|f_m| \leqslant 2/m$。

衰减的证明

角度 θ_i 分布在 $[\alpha, \beta]$ 上，$\beta = \theta_1 > \theta_2 > \cdots > \theta_{d/2} = \alpha$。当 $d \to \infty$、角度间隔足够小且均匀时，离散求和可近似为积分：

$$f_m \approx \int_{\alpha}^{\beta} \cos(mx)\,\mathrm{d}x \tag{4-67}$$

$$|f_m| \approx \left| \int_{\alpha}^{\beta} \cos(mx)\,\mathrm{d}x \right| = \left| \left[\frac{1}{m} \sin(mx) \right]_{\alpha}^{\beta} \right| = \frac{1}{m} \left| \sin(m\beta) - \sin(m\alpha) \right| \tag{4-68}$$

式 (4-68)：$\cos(mx)$ 的原函数为 $\frac{1}{m}\sin(mx)$，代入上下限作差。

$$|f_m| \leqslant \frac{2}{m} \tag{4-69}$$

正弦值域为 $[-1, 1]$，故 $|\sin(m\beta) - \sin(m\alpha)| \leqslant 2$；当 $m \to \infty$ 时 $2/m \to 0$，因此 $|f_m| \to 0$。证毕。

- 结论：f_t 的绝对值随相对距离增大而震荡衰减并趋近 0，注意力天然偏向较近的上下文。
- 但衰减的非单调性（图 4.13）会给学习长距离依赖带来困难，增加长文本建模的难度。

依据 《大语言模型：训练、精调与对齐》第 4.4.2 节（式 (4-67)–(4-69)）

频率与周期的关系

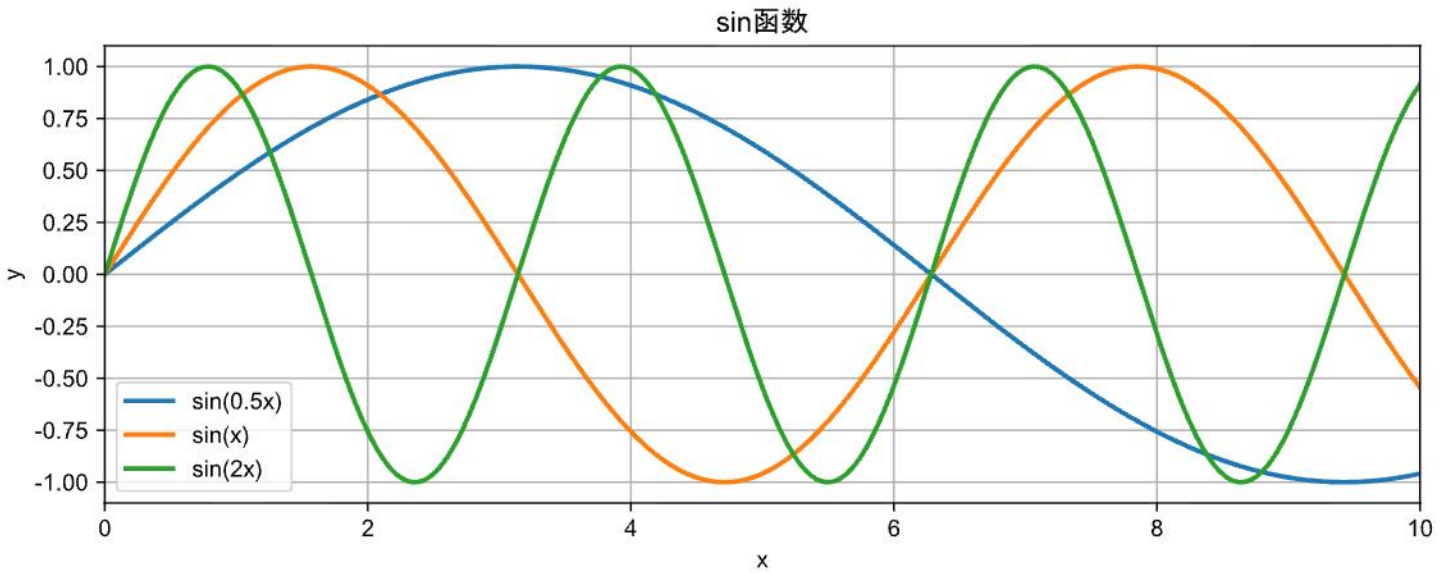


图 4.14　正弦函数周期与频率的关系：周期越短，频率越高

- 绝对位置编码与 RoPE 的元素都由周期函数算得。
- $\sin(\omega x)$ 的周期为 $2\pi/|\omega|$：ω 增大，周期变短、频率升高。
- 图 4.14 对比 $\omega = \frac{1}{2}, 1, 2$ 三种情形：波形从左到右依次变密。

这就是理解“高频组 / 低频组”的起点：RoPE 的不同维度对应不同的 ω。

各维度的频率分布

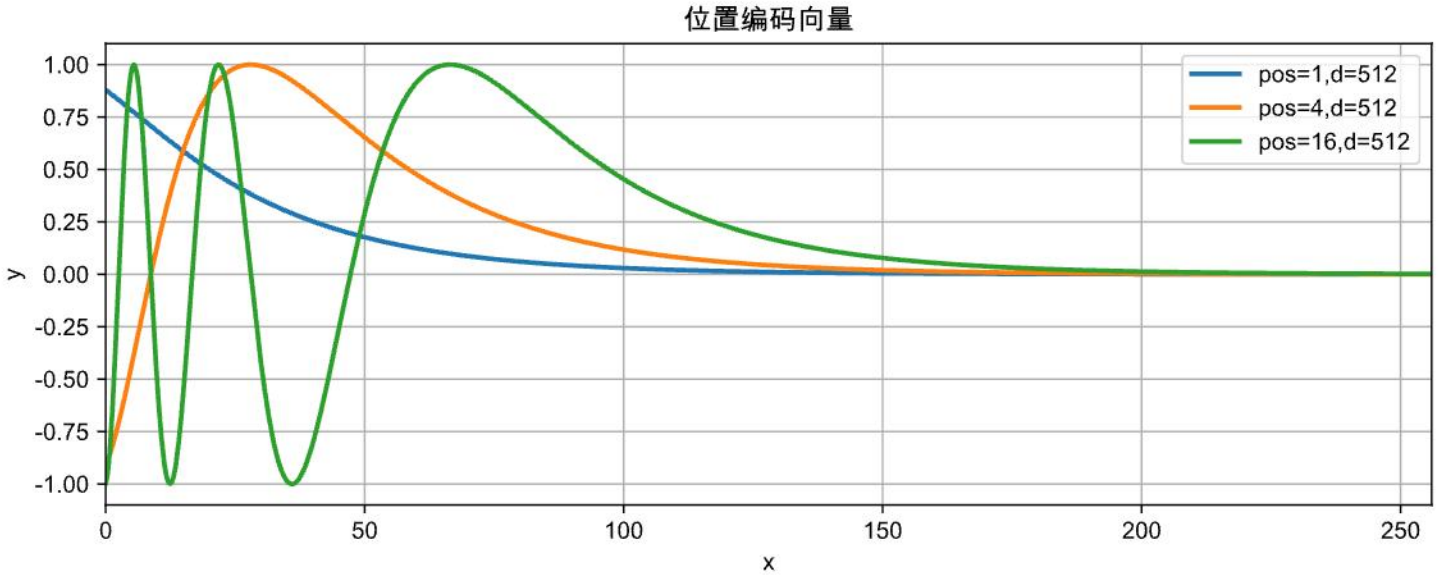


图 4.15　位置编码向量随维度的变化图

- RoPE 中旋转角度 $\theta_i = b^{-2(i-1)/d}$ 随维度 i 增加而变小。
- 向量左侧为低维信号，对应高频信息；右侧为高维信号，对应低频信息。
- 图 4.15 画出位置 $\{1, 4, 16\}$ 的编码向量：低维变换更快，高维变化更缓。

直觉　高频维负责区分相邻位置的细微差别，低频维负责刻画长程的位置关系。

分频段的衰减特性

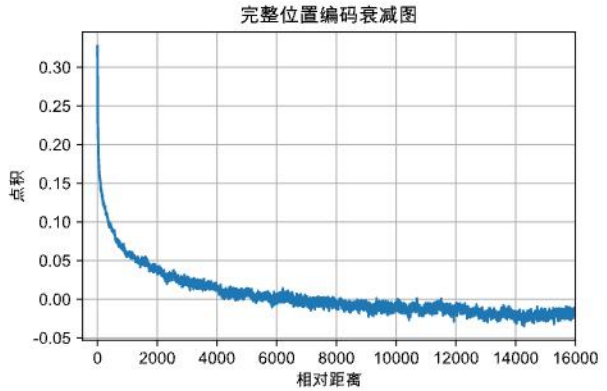


图 4.16(a)　全部维度合起来：连续下降后在 0 附近小幅振荡

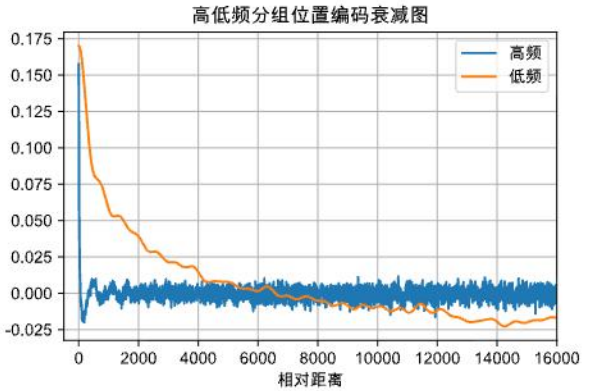


图 4.16(b)　按维度分成两半：高频组迅速衰减并振荡，低频组持续缓慢下降

把经 RoPE 变换的向量按维度分成两半，分别作为低频组与高频组，各自计算点积。

- 高频组：短距离上衰减更好；长距离时点积收敛到一个区间，并出现周期性振荡。
- 低频组：长距离下能保持更稳定的衰减。
- 这解释了两者互补：短程区分靠高频，长程建模靠低频。

位置内插（PI）

位置编码内插（Position Interpolation，PI）用于扩展上下文窗口。直接外推——让模型处理超出预训练长度的输入——会让位置编码值越出模型见过的数值分布，注意力分布因此混乱或过度集中于局部片段。

$$R(m') = R(m/s) \tag{4-70}$$

$s > 1$ 为内插系数，m 为原始位置索引，m' 为压缩后的新位置。PI 的核心思想是对位置索引压缩，把超长位置映射回预训练时见过的范围。

例 1

$s = 2$ 时，原始位置 $m = 2$ 映射到新位置 $m' = 1$。

例 2

$m = 17$、$s = 2$ 时，其位置编码的函数曲线与原始位置 $m = 8$ 的曲线接近（图 4.17）。

依据 《大语言模型：训练、精调与对齐》第 4.4.4 节（式 (4-70)）

PI 前后的曲线对比

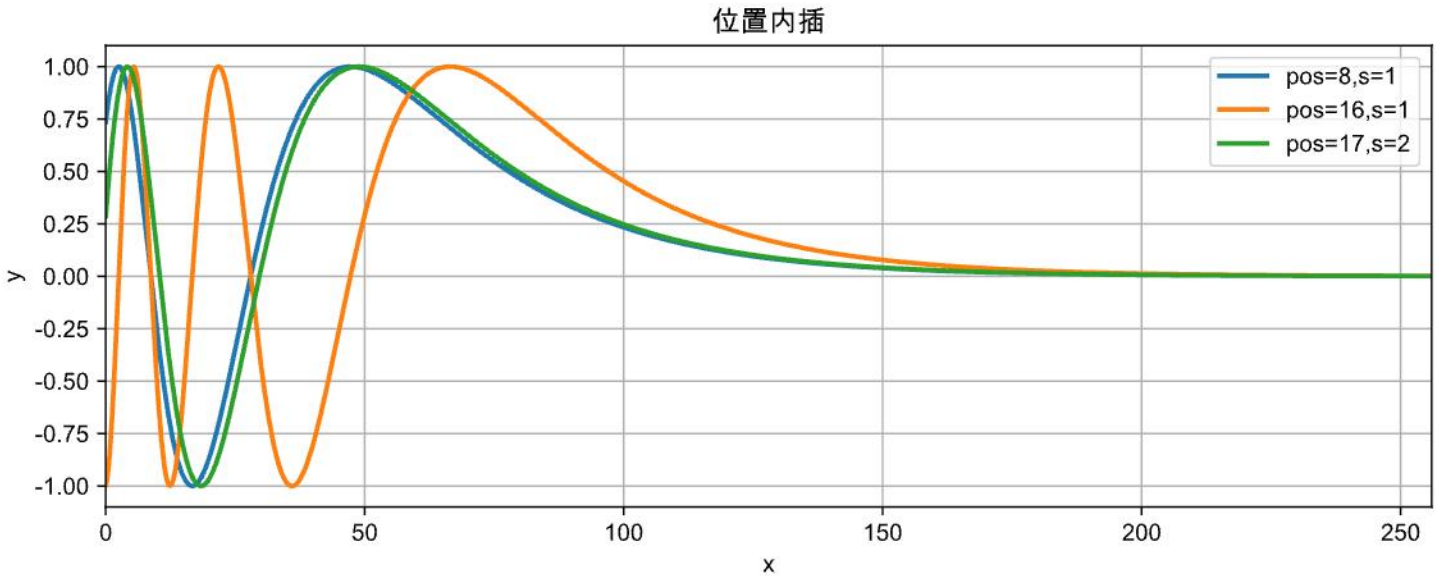


图 4.17　位置编码内插前后对比

- 原始位置 $m = 17$、内插系数 $s = 2$ 后，位置编码对应的函数曲线与原始位置 $m = 8$ 的曲线接近。
- 新位置落在模型预训练时见过的数值范围内，注意力分数不再异常。
- 代价：所有位置都被“挤”到一起，位置之间的间隔变小。

PI-RoPE 的特性

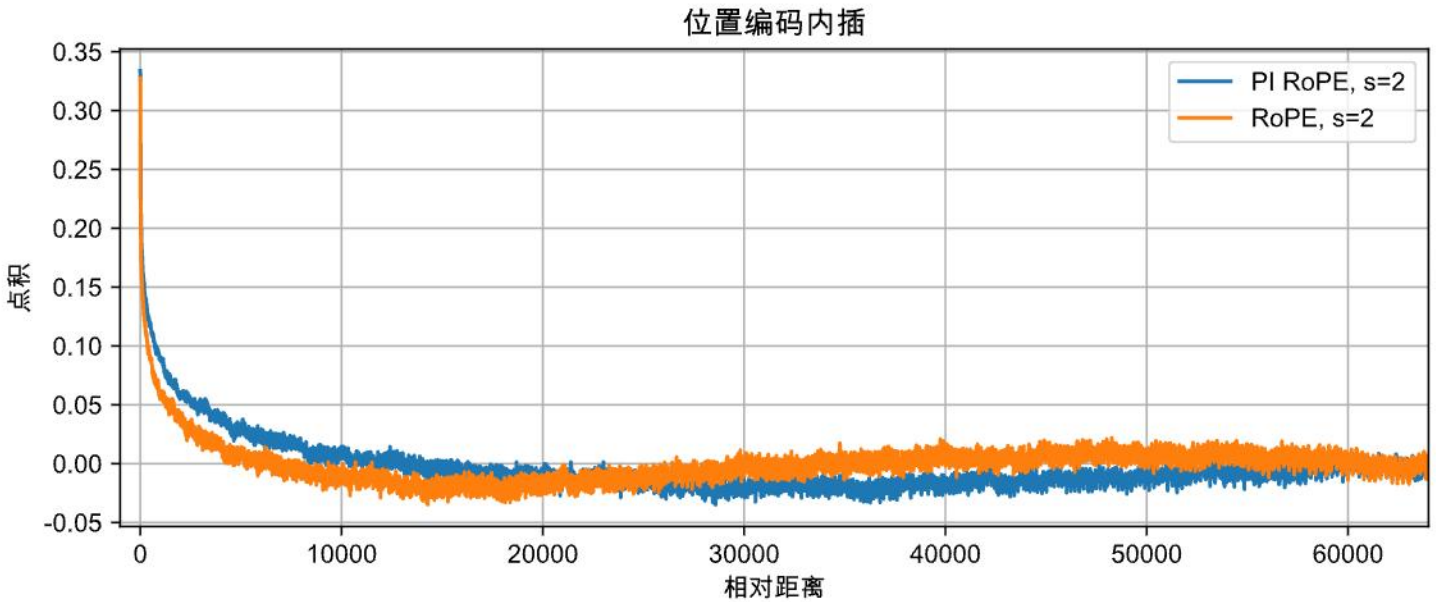


图 4.18　标准 RoPE 与 PI-RoPE 的注意力分数随相对位置衰减曲线对比图

- 近距（0–32k）：PI-RoPE 衰减比标准 RoPE 更慢，近距离的相对位置表达被削弱。
- 远距（32–64k）：PI-RoPE 衰减程度优于标准 RoPE，长距离信息保留更好。
- 短距离曲线更平缓，对应高频特征上的敏感性下降。

PI 损失近程精度的原因

PI 作用于周期性的位置编码函数 $\sin(\omega x)$ 时，相当于对所有频率分量做等比例压缩：

$$\omega \to \omega / s$$

内插系数 $s > 1$，每个频率分量都按同一比例降低——这是"等比例"的含义。

低维 = 高频

负责捕捉细微的位置变化。

频率被压缩后，对近距离位置的精细区分能力下降。

高维 = 低频

负责刻画较长程的位置关系。

变化相对较小，远距离感知得到增强。

净效果：远程能力提升，但近程精细位置信息受损，进而影响局部依赖关系的理解——这限制了 PI 扩展超长上下文时的最终效果，也促使 NTK-RoPE 采用非等比例的缩放。

NTK-RoPE

NTK 理论指出，高频分量易于学习但容易衰减或崩溃。NTK-RoPE 借鉴 PI 的思想，但改用非线性频率缩放：对高频分量（大维度索引、小旋转角）少缩放或不缩放，保留短距离的精细分辨力。

$$\left(\sin\left(\frac{m}{b'^{\frac{2k}{d}}}\right), \cos\left(\frac{m}{b'^{\frac{2k}{d}}}\right)\right) = \left(\sin\left(\frac{m}{(10\cdot b)^{\frac{2k}{d}}}\right), \cos\left(\frac{m}{(10\cdot b)^{\frac{2k}{d}}}\right)\right) \tag{4-71}$$

$$= \left(\sin\left(\frac{m}{10^{\frac{2k}{d}}\cdot b^{\frac{2k}{d}}}\right), \cos\left(\frac{m}{10^{\frac{2k}{d}}\cdot b^{\frac{2k}{d}}}\right)\right) \tag{4-72}$$

实现方式是用 $s>1$ 放大基数 $b'=sb$，这里以 $s=10$ 为例。把 $(10b)^{2k/d}$ 拆成 $10^{2k/d}\cdot b^{2k/d}$：缩放强度由 $10^{2k/d}$ 决定，随维度 k 变化。

- 低维度处（k 小）$10^{2k/d}$ 接近 1，旋转角几乎不变——高频分量被保留。
- 随维度增加，底数增大会使高维分量接近位置插值的形式：$m \to m/10^{2k/d} \approx m/10$（当 $k=2(d-1)$）。

依据 《大语言模型：训练、精调与对齐》第 4.4.5 节（式 (4-71)、(4-72)）

NTK-RoPE 的基数调整

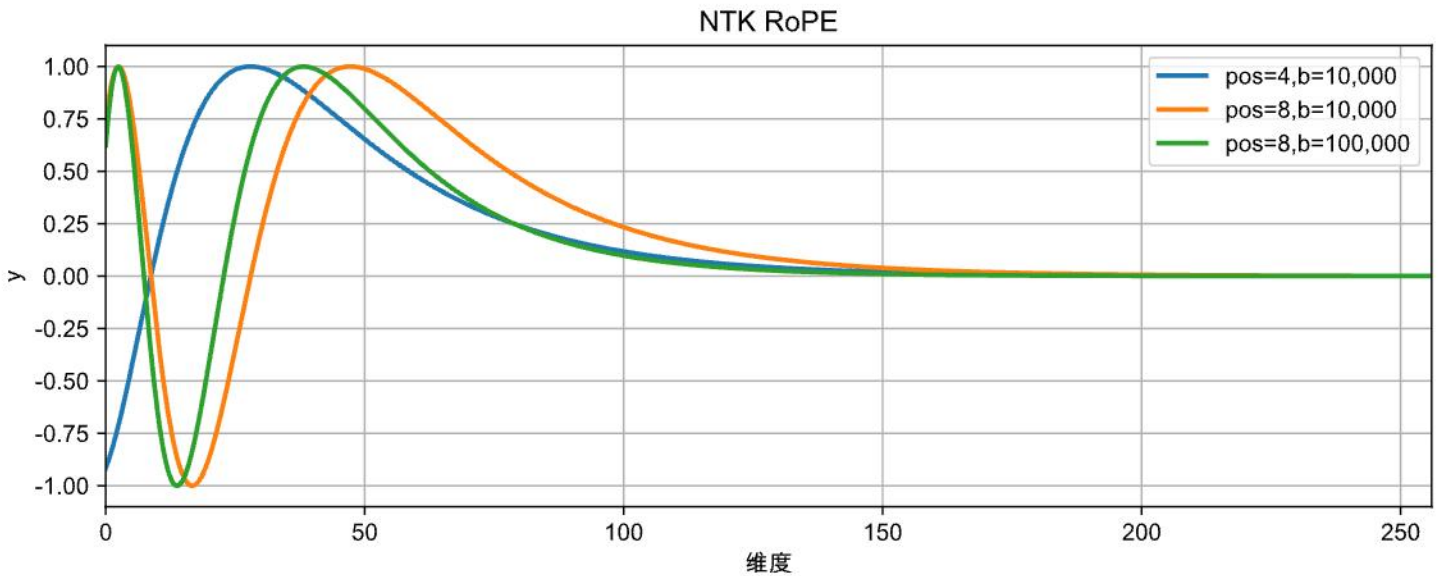


图 4.19　放大基数对位置编码向量影响

- 对比基数放大前后，高频分量的内插强度较小，低频分量的内插强度较大。
- 从左到右（维度由低到高），橙色线与绿色线的横向差距逐渐变大。
- 这正是“非线性缩放”：不是所有频率都被同等压缩。

64k 场景下的衰减特性

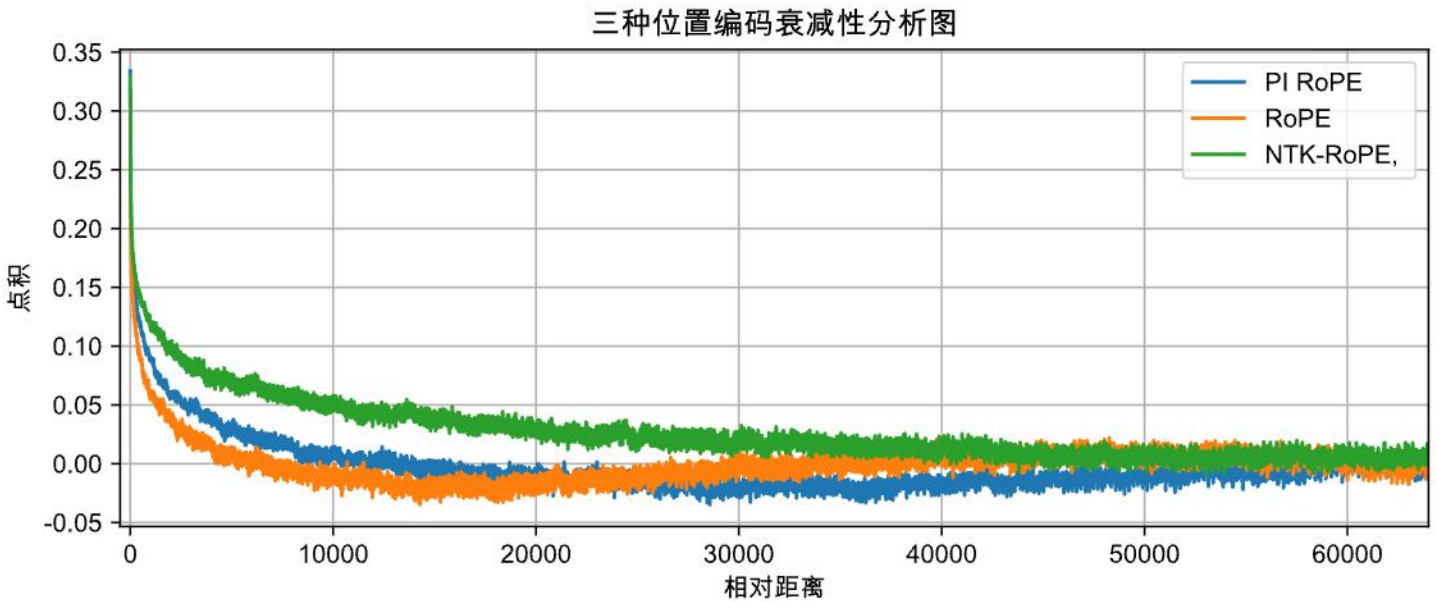


图 4.20　三种位置编码的衰减性可视化

- 测试结果表明，NTK-RoPE 的位置编码衰减特性稳定。
- 在 64k 的上下文长度场景中，衰减特性符合预期。
- 对比标准 RoPE 与 PI，NTK-RoPE 同时照顾了远程衰减与近程分辨。

两个频段的对比

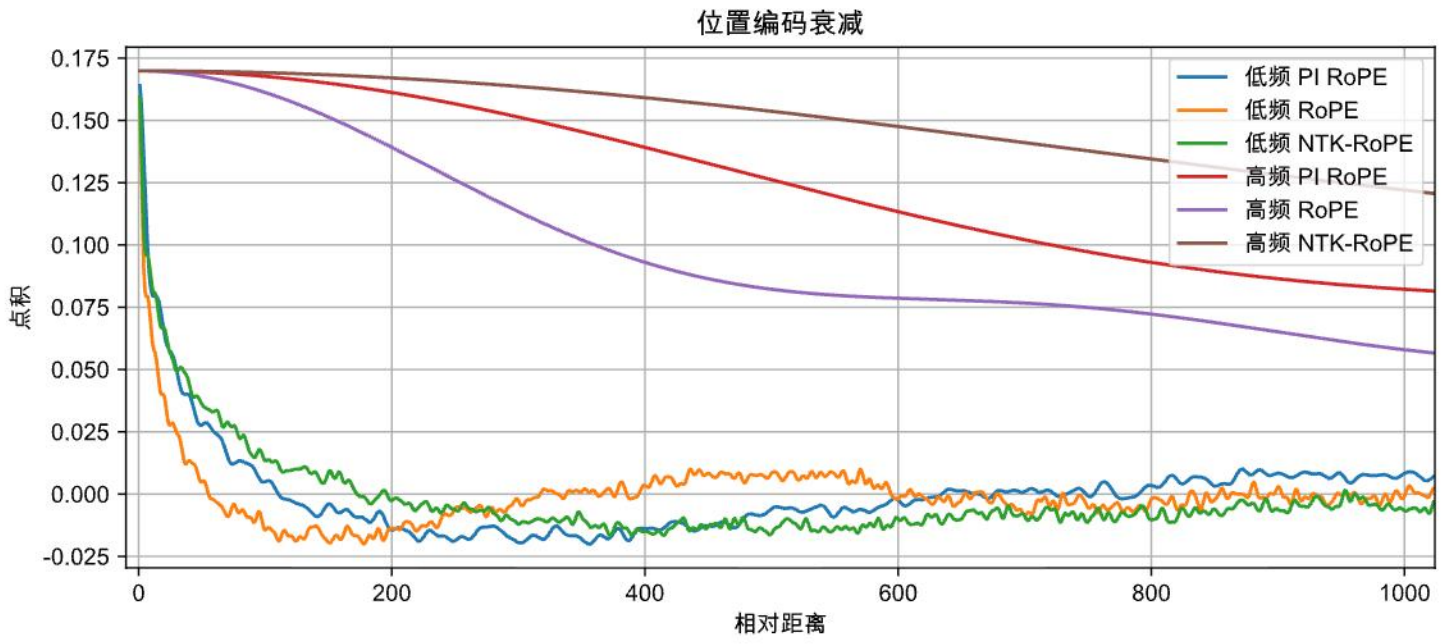


图 4.21　高低频位置编码的衰减性分析

- 为区分高频与低频分量的影响，分别对比 RoPE、PI、NTK-RoPE 在不同频段上的衰减。
- 结果显示，NTK-RoPE 在高频与低频区域都表现出符合理论预期的衰减行为。
- 相比 PI 的等比例压缩，NTK-RoPE 的频段表现更均衡。

长文本能力的评测方法

长文本扩展训练通常用 PPL 评估扩展效果；要考察模型在任意位置抓取信息的能力，还需专门的检索类基准。

指标 1

困惑度 PPL

在长文本语料上计算，衡量模型对新长度分布的适应程度。

指标 2

Needle in a Haystack

给定文本长度 L，把目标信息（“针”）放在相对位置 $y \in [0, 1]$ 处，测量检索准确率。

怎么读热力图 对不同的 (L, y) 组合测量检索准确率，可构建二维热力图（横轴文本长度，纵轴相对位置）。若某些位置出现明显低谷，说明模型在该相对位置的“回忆”能力不足，与衰减曲线的行为相互印证。

4.5 小结

本节内容

结构要点回顾

与后续章节的衔接

把 4.1–4.4 串起来：Llama 用四个模块替换做成一个纯解码器，KV-Cache 让推理可用，长文本继续预训练让上下文变长。

Llama 结构要点

Llama 是一种自回归大语言模型，Llama-3 的最大参数规模达到 70B；其网络结构用多种手段替换了标准 Transformer 组件。

归一化

RMSNorm

只重缩放，计算省、梯度稳。

位置

RoPE

旋转注入相对位置，底数可调。

前馈

SwiGLU

门控激活增强特征表达。

注意力

GQA

减少投影参数与 KV-Cache 显存，注意力计算量不变。

- Llama-3 在多个基准上表现出色：8B 模型经 15T 词元训练后追平 Llama-2 70B。
- KV-Cache 预填充建缓存、解码只算新词元（4.3）；长文本靠继续预训练改造位置编码（4.4）。

Llama 的后续发展

生态

微调与多模态

Alpaca 对预训练模型做指令微调；LLaVA 将其扩展为多模态版本。

架构演进

MoE 替换前馈层

Mixtral、DeepSeek 系列用 MoE 替换标准前馈层，扩展参数规模并提升计算效率（详见 4.6）。

沿"如何把注意力做得更省"这条线，衍生出多种预训练模型；它们与 Llama 的纯解码器思路并列，各有取舍：

Linear Attention

线性注意力

Performer、Linformer：用核函数或低秩投影改写 Softmax 注意力，把复杂度从 $O(L^2)$ 降到 $O(L)$，代价是表达力弱于全注意力。

Sparse Attention

稀疏注意力

Longformer、BigBird：只保留局部窗口 + 少量全局词元，长文档任务上以更少的连接覆盖全序列。

Diffusion LLM

扩散式语言模型

LLaDA、Dream：用去噪扩散代替自回归逐词元生成，支持并行解码与双向上下文，采样步数与质量需权衡。

4.6

附录：稀疏混合专家系统

Llama-4 改用 MoE：用门控为每个词元只挑 Top-K 个专家，在扩大参数规模的同时控制计算量。

本节内容

稀疏 MoE 的计算

负载均衡损失

稀疏混合专家（MoE）

混合专家系统（MoE）是一种集成学习系统，包含门控与多个参数独立的专家；门控给出各专家对当前预测的重要性。

稠密 MoE

每个专家都参与计算。

算力消耗过多，失去了稀疏激活带来的效率优势。

稀疏 MoE

门控只取 Top-K 个专家参与计算。

计算量随 K 而非 N 增长，可在大参数规模下保持推理效率。

> **专家** 在大语言模型中，一个前馈层或一个 SwiGLU 都可以作为一个专家 $E_i(\cdot)$。设共有 N 个专家，门控用线性层 $\boldsymbol{W}_{\text{gate}} \in \mathbb{R}^{d \times N}$ 实现。

稀疏 MoE 的计算步骤

$$\boldsymbol{s}_t = \boldsymbol{x}_t \boldsymbol{W}_{\text{gate}} \tag{4-73}$$

$$s'_{t,i} = \begin{cases} s_{t,i}, & s_{t,i} \in \texttt{Topk}(\{s_{t,j} \mid 1 \leqslant j \leqslant N\}, K), \\ -\infty, & \text{otherwise} \end{cases} \tag{4-74}$$

$$\boldsymbol{g}_t = \texttt{Softmax}(\boldsymbol{s}'_t) \tag{4-75}$$

$$\boldsymbol{y}_t = \sum_{i=1}^{N} g_{t,i} E_i(\boldsymbol{x}_t) \tag{4-76}$$

$\boldsymbol{x}_t \in \mathbb{R}^d$ 为第 t 个词元的特征，$\boldsymbol{s}_t \in \mathbb{R}^N$ 为各专家分数；未入选专家的分数置为 $-\infty$，Softmax 后其权重为 0；$E_i(\boldsymbol{x}_t) \in \mathbb{R}^d$。

- 式 (4-74) 的 $-\infty$ 与注意力掩码同理：Softmax 后权重恰为 0，未选中的专家不贡献输出。
- 式 (4-76) 只对被选中的 K 项有非零权重，故实际只前向 K 个专家。
- 一个序列的多个词元会按门控结果派发到不同专家：门控逐词元分配，每个词元激活自己的 Top-K。

依据 《大语言模型：训练、精调与对齐》第 4.6 节（式 (4-73)–(4-76)）

负载均衡损失

如果某些专家在初始阶段就被频繁选中，训练会一直偏向它们，其余专家始终得不到激活，自适应分配就失效了。为此引入负载均衡损失。

$$f(i) = \frac{1}{T} \sum_{t=1}^{T} \mathbb{I}\left(s_{t,i} \in \mathbf{Topk}\left(\{s_{t,j} \mid 1 \leqslant j \leqslant N\}, K\right)\right) \tag{4-77}$$

$$g(i) = \frac{1}{T} \sum_{t=1}^{T} g_{t,i} \tag{4-78}$$

$$\mathcal{L}_{\text{load}} = \alpha \sum_{i=1}^{N} f(i)\, g(i) \tag{4-79}$$

$f(i)$：词元被分配到专家 i 的比率；$g(i)$：专家 i 的门控权重均值；α 为损失权重。

- 两项相乘再求和：某专家被选得越多、权重越大，惩罚越大，从而把词元推向冷门专家。
- 对于包含 MoE 网络的大型模型，还可以通过专家并行（Expert Parallel）提高并行性（见第 9 章）。

依据 《大语言模型：训练、精调与对齐》第 4.6 节（式 (4-77)–(4-79)）

思考题与实践题

1 Llama-3 有什么样的缩放定律结论？结合 15T 词元与 8B 追平 70B 讨论。

2 写出式 (4-2)–(4-7)，说明两处残差连接的作用。

3 从系统角度，推理时预填充和解码阶段 KV-Cache 各有什么特点？

4 GQA 中 $h = 32$、$g = 8$ 时，$\boldsymbol{W}_k$、$\boldsymbol{W}_v$ 的维度是多少？KV-Cache 相对 MHA 降为几分之几？

5 相较绝对位置编码，RoPE 为什么更优？从式 (4-28)–(4-31) 的四个交叉项出发说明。

6 写出无 KV-Cache 与有 KV-Cache 时生成第 $t+1$ 个词元的注意力计算，并比较计算量。

7 GQA 多头切分、RoPE 变换、KV-Cache 存储的顺序是什么？说清每一步作用在哪个张量上。

8 如何实现免训练的长度外推？以 PI、NTK-RoPE 与 YaRN 为例比较做法与代价。